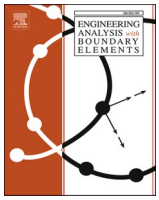
Engineering Analysis with Boundary Elements 184 (2026) 106653



Contents lists available at ScienceDirect

Engineering Analysis with Boundary Elements

journal homepage: www.elsevier.com/locate/enganabound





Tanh-B6 KAN-based PINN for solving thin plate bending problems

Shuo Liu, Yonghong Cao *

School of aerospace engineering, North University of China, 3 Xueyuan Road, Taiyuan, 030051, Shanxi, China

ARTICLE INFO

Keywords:
Physics-Informed neural networks
Kolmogorov-Arnold networks
B-spline
Governing equations of Kirchhoff-Love plate theory
Partial differential equations

ABSTRACT

Solving high-order partial differential equations, such as the governing equations of Kirchhoff thin plate bending, remains a significant challenge for traditional Physics-Informed Neural Networks (PINNs). Conventional Multi-Layer Perceptron (MLP) architectures often suffer from spectral bias and gradient instability when computing high-order derivatives. To address these limitations, this study introduces a novel framework: the Tanh-B6 KAN-based PINN. This method integrates Kolmogorov-Arnold Networks (KAN) featuring learnable Tanh activation functions and sixth-order B-spline basis functions into the PINN architecture. Specifically, the sixth-order B-splines ensure C^4 continuity, providing stable analytical computation for high-order derivatives, while the Tanh activation captures global trends. The effectiveness of this approach is validated through comprehensive numerical experiments on elliptical, triangular, rectangular, and L-shaped thin plates subject to varying boundary and load conditions. Comparative results demonstrate that the Tanh-B6 KAN-based PINN significantly outperforms the traditional MLP-PINN, reducing the relative L_2 norm error and Mean Absolute Error (MAE) of the displacement field and boundaries by one to three orders of magnitude, while reducing the number of parameters by two to three orders of magnitude. The proposed method offers a robust, interpretable, and highly efficient solution for high-order mechanics problems.

1. Introduction

With the development of modern science and technology, many thin plate structures, such as steel plates, elastic foundations, and plastic plates, have been widely applied in civil engineering, hydraulic engineering, highways, and railways. The thin plate bending governing equation established based on Kirchhoff's thin plate theory is a fourth-order partial differential equation. Although this PDE can accurately describe the mechanical behavior of thin plates, obtaining an analytical solution is often difficult in practical engineering applications, particularly when encountering complex geometric shapes or non-typical boundary conditions [1]. In such cases, numerical calculation methods become an effective means of solving such problems, such as the commonly used finite element method [2–5], mesh-free methods [6–11], finite difference method [12], finite volume method [13], and boundary element method [14]. Significant advancements have recently been made in these traditional domains. For instance, in the context of the finite element method, edge-based strain smoothing techniques [15] and generalized conforming elements based on area coordinates [16] have been developed to enhance accuracy and handle mesh distortion. Similarly, within mesh-free frameworks, cell-based smoothed radial point interpolation methods have been successfully applied to both lower and upper bound limit analysis of thin plates [17,18].

Recently, AI for PDE has emerged as an important direction within AI for Science, referring to a class of algorithms that utilize deep learning to solve partial differential equations (PDEs). The strategies for solving partial differential equations using deep learning include the strong-form and the weak-form. Physics-Informed Neural Networks (PINNs) [19] and the Deep Gauss-Riemann Method (DGM) [20] exemplify strong-form approaches, where the neural network is trained to satisfy the PDE residual and boundary conditions directly at collocation points. Conversely, weak-form methods, such as the Deep Ritz Method (DRM) [21] and the Deep Energy Method (DEM) [1,22], reformulate the PDE into a variational principle, which the network learns to minimize. For plate and shell mechanics analysis, Li et al. [23] established a neural network-based computational framework to characterize the finite deformation of elastic plates and compared methods based on partial differential equations and energy-based methods to construct the loss function. Zhuang et al. [24] proposed an energy-based method (DAEM) based on deep autoencoders for bending, vibration and buckling analysis of Kirchhoff plates, and employed transfer learning to further improve efficiency and generalizability. Based on Naghdi shell theory,

* Corresponding author.
E-mail address: caoyonghong@nuc.edu.cn (Y. Cao).

https://doi.org/10.1016/j.enganabound.2026.106653
Received 19 November 2025; Received in revised form 13 January 2026; Accepted 18 January 2026
Available online 23 January 2026
0955-7997/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Bastek et al. [25] proposed a PINN to predict small strain responses of arbitrarily curved shells in non-Euclidean domains. Furthermore, several recent works have specifically focused on applying PINNs to thin plate bending problems. Wang and Thai [26] developed a PINN framework for laminated composite plates under bending. Wang et al. [27] introduced Chien-physics-informed neural networks for singularly perturbed boundary-layer problems, which can be relevant to thin plate analysis. Peng et al. [28] investigated the bending analysis of thin plates with variable stiffness resting on elastic foundations using a two-network strategy PINN. Zhang and He [29] proposed multi-scale-matching neural networks for the thin plate bending problem, and Kafkas et al. [30] explored solving linear elasticity problems using PINNs, which can be extended to plate bending.

Recently, Liu et al. proposed the Kolmogorov-Arnold Networks (KAN) [31]. The core of KAN lies in the introduction of a composite function framework that can learn activation functions. In the original KAN, B-splines were used to construct the activation function due to their excellent fitting ability [31]. Subsequently, some studies modified the B-splines used as activation functions in KAN, replacing them with Chebyshev orthogonal polynomials [32,33], radial basis functions [34], and wavelet transforms [35]. Function approximation theory in numerical analysis indicates that different weight functions produce different orthogonal polynomials, each with its unique advantages. Theoretically, almost all traditional approximation functions can replace B-splines in KAN (Kolmogorov-Arnold Networks), making the framework highly versatile. Building upon the concept of KANs, several physics-informed approaches have emerged. Guo et al. [36] introduced Physics-informed Kolmogorov-Arnold Networks (PIKAN) utilizing Chebyshev polynomials for fluid mechanics. Similarly, Wang et al. [37] proposed Kolmogorov-Arnold-Informed Neural Networks (KINN) as a physics-informed deep learning framework for solving both forward and inverse problems. These works highlight the growing interest in integrating KANs with Physics-Informed Neural Networks.

Therefore, we propose to use the Tanh-B6 KAN-based PINN method to solve the fourth-order Kirchhoff thin plate bending problem. This work primarily involves introducing Kolmogorov-Arnold Networks (KAN) featuring learnable Tanh activation functions and sixth-order B-spline basis functions into the PINN framework to solve Kirchhoff thin plate bending problems. Our study systematically compares the accuracy and efficiency of Tanh-B6 KAN-based PINN against traditional MLP-PINN specifically for the challenging fourth-order thin plate bending problem. Our motivation is predicated on the limitations of conventional Multi-Layer Perceptron (MLP) when utilized within Physics-Informed Neural Network (PINN) frameworks. MLPs are often characterized by high parameter redundancy and an inherent lack of interpretability, which can lead to compromised accuracy and model opacity, particularly for complex physical systems. In contrast, Kolmogorov-Arnold Networks (KAN) offer superior parameter efficiency. We introduce a novel KAN-based architecture that integrates learnable Tanh activation functions with sixth-order B-spline basis functions. The functional construction of this Tanh-B6 KAN is designed to more intrinsically align with the numerical principles underlying the solution of high-order Partial Differential Equations (PDE). Based on this, we propose the hypothesis that replacing traditional MLP with the Tanh-B6 KAN-based architecture when solving thin plate bending problems using PINNs will significantly enhance performance.

The structure of this paper is organized as follows. In Section 2, we present the nondimensionalized fourth-order governing partial differential equation and boundary conditions based on Kirchhoff plate theory. Section 3 provides a detailed description of our method, including the KAN (Kolmogorov-Arnold Networks) theory, the construction of the Tanh-B6 KAN-based PINN architecture, loss functions, and the algorithmic process. Section 4 validates the method through four numerical experiments on plates with different geometries and loads, presenting a comprehensive comparative analysis of MLP-PINN, standard B6 KAN-based PINN, and the proposed Tanh-B6 KAN-based PINN. Section 5 investigates the influence of network architecture parameters, specifically conducting an ablation study on the necessity of the node bias term and analyzing the impact of the number of neurons and hidden layers on convergence performance. Finally, Section 6 summarizes our current work.

2. Governing equations

Considering an isotropic thin plate with thickness h, Poisson's ratio ν, and elastic modulus E, the governing differential equation of Kirchhoff's thin plate theory is:

$$D\nabla^4 w - q(x,y) = 0 \tag{1}$$

where w is the deflection of the thin plate, ∇^2 is the Laplace operator, $D = Eh^3/[12(1-\nu^2)]$ is the flexural rigidity of the plate, and $q(x, y)$ is the transverse load distribution.

To improve the training efficiency and numerical stability of the neural network, we apply physical nondimensionalization to the above equation. We introduce a characteristic length L (typically the maximum characteristic dimension of the plate) and a characteristic deflection W_0. The nondimensional coordinates and deflection are defined as follows:

$$x^* = \frac{x}{L}, y^* = \frac{y}{L}, w^* = \frac{w}{W_0} \tag{2}$$

By selecting the characteristic deflection as $W_0 = \frac{q_0 L^4}{D}$ (where q_0 is the amplitude of the characteristic load), and substituting it into Eq. (1), we obtain the nondimensionalized governing equation:

$$\nabla^{*4} w^* - \frac{q(x,y)}{q_0} = 0 \tag{3}$$

At this point, the coefficient of the high-order derivative term in the equation becomes 1, effectively avoiding numerical stiffness issues caused by the physical parameter D or geometric dimension L.

The three most common boundary conditions in elasticity theory are transformed into their nondimensional forms as follows:

For clamped boundaries, the boundaries are fully fixed, allowing neither transverse displacement nor angular rotation. The following should be considered:

$$w^* = 0, \frac{\partial w^*}{\partial n^*} = 0 \tag{4}$$

For simply supported boundaries, the boundary can rotate freely, but transverse displacement is restricted. Considering the influence of Poisson's ratio, the normal bending moment M_n is zero. The following should be considered:

$$w^* = 0, M_n^* = -\left(\frac{\partial^2 w^*}{\partial n^{*2}} + \nu \frac{\partial^2 w^*}{\partial t^{*2}}\right) = 0 \tag{5}$$

For free boundaries, where no bending moment M_n or equivalent shear force V_n acts at the boundary, free deformation is allowed. The following should be considered:

$$M_n^* = 0, V_n^* = Q_n^* + \frac{\partial M_{nt}^*}{\partial t^*} = 0 \tag{6}$$

where n^* and t^* represent the nondimensional coordinates along the normal and tangential directions of the boundary, respectively. The nondimensional equivalent shear force V_n^* consists of the nondimensional transverse shear force Q_n^* and the derivative of the nondimensional twisting moment M_{nt}^* with respect to the nondimensional tangential coordinate t^*.

3. Tanh-B6 KAN-based PINN

3.1. Kolmogorov–Arnold networks theory

The Kolmogorov-Arnold representation theorem provides a theoretical foundation for approximating multivariate functions. According to the theorem, any continuous function $f(x) = f(x_1,x_2,\cdots x_n)$ defined in an n-dimensional compact domain can be decomposed into a two-layer linear combination of a finite number of univariate functions:

$$f(x) = f(x_1, x_2, \cdots x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right) \tag{7}$$

where both the outer functions $\Phi_q(\cdot)$ and the inner functions $\phi_{q,p}(\cdot)$ are continuous univariate functions.

This theorem breaks the traditional curse of dimensionality [38,39], demonstrating that the complexity of high-dimensional functions can be approximated by the superposition of univariate functions, providing a mathematical guarantee for Kolmogorov-Arnold networks.

The output of the $j-th$ neuron in the $l-th$ layer of the traditional MLP is:

$$z_j^l = \sigma \left(\sum_{k=1}^{n_{l-1}} w_{jk}^l \cdot z_k^{l-1} + b_j^l \right) \tag{8}$$

where z_j^l is the output of the $j-th$ neuron in the $l-th$ layer, w_{jk}^l is the weight of the $k-th$ neuron in the $(l-1)-th$ layer transmitted to the $j-th$ neuron in the $l-th$ layer, b_j^l is the bias term of the $j-th$ neuron in the $l-th$ layer, and σ is a fixed nonlinear activation function (such as Tanh, Sigmoid or ReLU).

KAN departs from the traditional fixed activation function paradigm of MLP by adopting learnable univariate functions ϕ_{jk}^l to replace the combination of "linear weight matrices plus fixed activation functions". The nonlinearity in KAN is directly introduced by these learnable univariate functions, enabling it to theoretically achieve higher accuracy with fewer parameters. The output of the $j-th$ neuron in the $l-th$ layer of KAN is:

$$z_j^l = \sum_{k=1}^{n_{l-1}} \phi_{jk}^l \left(z_k^{l-1} \right) + \beta_j^l \tag{9}$$

where ϕ_{jk}^l is a learnable univariate function defined on the edge $k \to j$ that connects the $k-th$ neuron in the $(l-1)-th$ layer to the $j-th$ neuron in the $l-th$ layer. β_j^l is a node bias term. These edge functions ϕ_{jk}^l are typically parameterized by a set of basis functions (such as B-splines, Chebyshev polynomials, or radial basis functions), allowing them to be optimized during training.

In the Tanh-B6 KAN architecture proposed in this study, we deliberately omit the explicit node bias term β_j^l. While bias terms are fundamentally important in general neural network theory for adjusting activation thresholds, the B-spline basis functions employed in our architecture possess the Partition of Unity property [40], which allows them to implicitly represent constant shifts. A detailed theoretical proof and experimental verification regarding the unnecessariness of the bias term in this specific architecture are provided in Section 5.1. Consequently, the expression for the neuron output is simplified to:

$$z_j^l = \sum_{k=1}^{n_{l-1}} \phi_{jk}^l \left(z_k^{l-1} \right) \tag{10}$$

In this study, to solve the fourth-order Kirchhoff plate bending problem, we propose using learnable *Tanh* activation functions and sixth-order B-spline basis functions to explicitly model the learnable activation functions ϕ_{jk}^l:

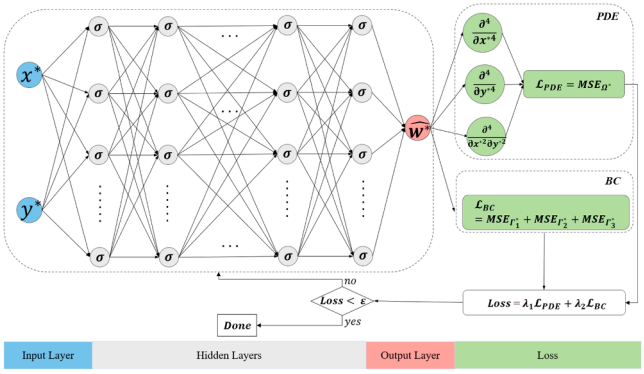


Fig. 1. MLP-PINN neural network architecture.

$$\phi_{jk}^l(x) = \underbrace{a_{jk}^l \cdot Tanh\left(b_{jk}^l \cdot x\right)}_{\text{Learnable Tanh basis}} + \underbrace{\sum_{i=1}^{G} c_{jk,i}^l B_{i,6}(x)}_{\text{Learnable sixth-order B-spline basis}} \tag{11}$$

where $Tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$. a_{jk}^l and b_{jk}^l are learnable parameters that control the amplitude and scale of the *Tanh* function, respectively, allowing it to adaptively adjust its shape to match data features. $B_{i,6}(x)$ is the $i-th$ sixth-order B-spline basis function, which possesses C^4 continuity. This is crucial for solving the fourth-order Kirchhoff plate problem, as it ensures the existence and smoothness of higher-order derivatives. $c_{jk,i}^l$ are the learnable coefficients corresponding to the $i-th$ B-spline basis function. G is the number of B-spline basis functions, which is typically determined by the number of pre-defined grid intervals. By summing these two components, $\phi_{jk}^l(x)$ gains the global stability of *Tanh* and the local high precision of B-splines, enabling it to efficiently approximate complex high-dimensional functions and their derivatives with fewer parameters.

3.2. B-spline function

B-spline functions are the core tool for parameterizing single-variable functions in KAN. Given a node vector $\mathbf{T} = \{t_0, t_1, \ldots, t_N\}$, where $t_i \le t_{i+1}$, the recursive definition of the m-th-order B-spline basis function $B_{i,m}(x)$ is:

$$\begin{aligned} B_{i,1}(x) &= \begin{cases} 1, & t_i \le x < t_{i+1} \\ 0, & \text{else} \end{cases} \\ B_{i,m}(x) &= \frac{x - t_i}{t_{i+m-1} - t_i} B_{i,m-1}(x) + \frac{t_{i+m} - x}{t_{i+m} - t_{i+1}} B_{i+1,m-1}(x) \end{aligned} \tag{12}$$

The B-spline is non-zero only within the interval $[t_i, t_{i+m})$ and possesses C^{m-2} continuity [40], making it suitable for higher-order derivative calculations. Specifically, the sixth-order B-spline function exhibits continuity C^4, which means that it is continuous up to the fourth derivative. Its explicit expression is a piecewise quintic polynomial. The construction of this function follows a recursive definition: the initial condition is set as $B_6^{(-1)}(x) = 0$, and then the recursive formula for each interval is as follows:

$$B_6^{(k)}(x) = B_6^{(k-1)}(x) + \frac{(-1)^k \binom{6}{k} (x-k)^5}{120}, x \in [k, k+1) \tag{13}$$

where $k = 0, 1, 2, 3, 4, 5$.

3.3. Tanh-B6 KAN-based PINN network architecture

As shown in Fig. 1, the MLP-PINN (Multi-Layer Perceptron Physics-

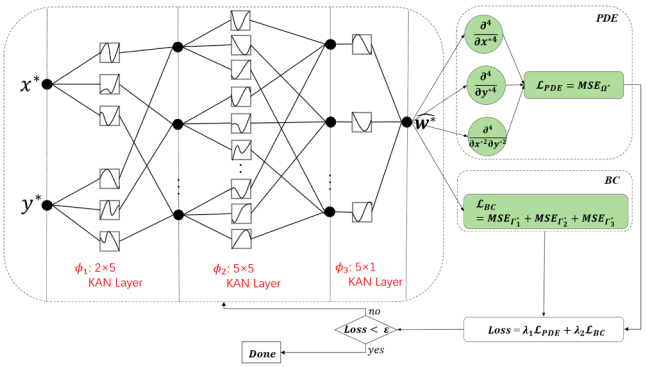


Fig. 2. Tanh-B6 KAN-based PINN network architecture.

Informed Neural Network) uses a fully connected neural network (MLP) as its underlying function. It solves partial differential equations by minimizing a loss function that includes the residual of the physical equation and the boundary conditions. The physics-based loss term (PDE residual) in PINN requires computing the derivatives of the network output $w^*(\mathbf{x})$ with respect to its input $\mathbf{x}$, such as $\partial w^*/\partial x^*$, $\partial^2 w^*/\partial x^{*2}$, and all the way up to $\nabla^{*4} w^*$. This process is accomplished through automatic differentiation, which is essentially a repeated application of the chain rule. An MLP can be represented as $w^*(\mathbf{x}) = L_N \circ \sigma \circ L_{N-1} \circ \cdots \circ \sigma \circ L_1(\mathbf{x})$, where L_i denotes a linear layer. Calculating high-order derivatives of a neural network's output with respect to its input, for example $\nabla^4 w^*$, involves repeatedly applying the chain rule. This process propagates through the network from the output layer back to the input layer. Crucially, for each layer traversed, the derivative is multiplied by the derivative of that layer's activation function, σ'. When using an activation function like the hyperbolic tangent (*Tanh*), its derivative, $sech^2(x)$, is always in the range (0, 1]. In a deep architecture, this means the final high-order derivative is the result of multiplying a long chain of numbers that are smaller than one. This successive multiplication causes the derivative's magnitude to decay exponentially, an issue known as the vanishing gradient problem. This attenuation of high-order derivative information makes it exceptionally difficult for the model to capture and resolve high-frequency features or regions of high curvature in the target solution. This issue is also known in academia as the "spectral bias" of Multi-Layer Perceptron [41], meaning that standard MLP tends to prioritize learning low-frequency functions, while learning high-frequency information is very slow and difficult. Since higher-order derivatives amplify the high-frequency parts of a function, MLP-PINN perform poorly when solving equations that require accurate high-order derivatives, especially in deep networks.

Tanh-B6 KAN-based PINN is based on the Kolmogorov-Arnold representation theorem and uses learnable Tanh activation functions and sixth-order adaptive B-spline basis functions to construct the network. It also solves partial differential equations by minimizing the loss from the physics equation residual and boundary conditions. The sixth-order adaptive B-spline basis functions in KAN are themselves C^4 continuous, and differentiating a B-spline results in a lower-order B-spline, which is an analytical, stable, and local operation [40]. For example, a sixth-order B-spline function is C^4 continuous. Unlike an MLP, it does not rely on a deep, global chain rule for computation. Therefore, calculating $\nabla^{*4} w^*$ is achieved simply by analytically differentiating the B-spline basis functions four times, completely avoiding the numerical instability issues caused by multiplying the derivatives of many activation functions. Furthermore, B-splines have local support [40], meaning each basis function is non-zero only within a small region of the input space. This allows KAN to capture the local features of a solution very well, much like the Finite Element Method (FEM), which is a weakness of MLP [41].

As shown in Fig. 2, all numerical examples in this paper use a Tanh-B6 KAN-based PINN network architecture of [2, 5, 5, 1]. The input vector for this network architecture is $\mathbf{x} \in \mathbb{R}^2$, and the three KANLinear layers are denoted as $f_1(2 \times 5)$, $f_2(5 \times 5)$, and $f_3(5 \times 1)$. The forward propagation process of the entire network can then be expressed in the following function composition form:

$$\mathbf{y} = f(\mathbf{x}) = f_3(f_2(f_1(\mathbf{x}))) \tag{14}$$

where the output vector $\mathbf{y} \in \mathbb{R}^1$ is the final output of the model.

3.4. Loss function

In Physics-Informed Neural Networks (PINNs), since we adopt the nondimensionalized equation, the construction of the loss function becomes more natural and numerically balanced. The network no longer directly predicts the physical deflection w, but instead predicts the nondimensional deflection $\widehat{w^*}((x^*, y^*); \mathbf{\Phi})$. For the Kirchhoff plate bending problem, the nondimensional PDE residuals and boundary condition residuals are as follows:

(1) Domain PDE Residual:

$$\nabla^{*4} \widehat{w^*}(x^*, y^*; \mathbf{\Phi}) - \frac{q(x^*, y^*)}{q_0} = 0, (x^*, y^*) \in \Omega^* \tag{15}$$

(2) Boundary Condition Residuals:

For the clamped boundary Γ_1^*, the following should apply:

$$\widehat{w^*}\left((x^*, y^*)_{\Gamma_1^*}; \mathbf{\Phi}\right) = 0, \frac{\partial \widehat{w^*}\left((x^*, y^*)_{\Gamma_1^*}; \mathbf{\Phi}\right)}{\partial n^*} = 0 \tag{16}$$

For the simply supported boundary Γ_2^*, the following should apply:

$$\widehat{w^*}\left((x^*, y^*)_{\Gamma_2^*}; \mathbf{\Phi}\right) = 0, M_n^*\left((x^*, y^*)_{\Gamma_2^*}; \mathbf{\Phi}\right) = \widetilde{M}_n^* \tag{17}$$

For the free boundary Γ_3^*, the following should apply:

$$M_n^*\left((x^*, y^*)_{\Gamma_3^*}; \mathbf{\Phi}\right) = 0, V_n^*\left((x^*, y^*)_{\Gamma_3^*}; \mathbf{\Phi}\right) = \widetilde{V}_n^* \tag{18}$$

where the effective shear force is defined as:

$$V_n^* = Q_n^*\left((x^*, y^*)_{\Gamma_3^*}; \mathbf{\Phi}\right) + \frac{\partial M_{nt}^*\left((x^*, y^*)_{\Gamma_3^*}; \mathbf{\Phi}\right)}{\partial t^*} \tag{19}$$

Where (x^*, y^*) are the coordinates of the data points, Ω^* is the elastic solution domain, and $\mathbf{\Phi}$ is the parameter set of the PINN. $\widetilde{M}_n^*$ and $\widetilde{V}_n^*$ are the prescribed bending moment and effective shear force applied at the boundary (usually zero for free edges). $M_n^*(x^*, y^*; \mathbf{\Phi}), Q_n^*(x^*, y^*; \mathbf{\Phi})$ and $M_{nt}^*(x^*, y^*; \mathbf{\Phi})$ are the nondimensional internal bending moment, shear force, and twisting moment, respectively. These mechanical quantities are derived from the PINN's nondimensional deflection prediction $\widehat{w^*}((x^*, y^*); \mathbf{\Phi})$ through nondimensionalized constitutive relations.

The mean square error (MSE) is used to measure the fitting error of PINN, and the loss function can be constructed as follows.

$$C(\mathbf{\Phi}) = \lambda_1 \mathscr{L}_{PDE} + \lambda_2 \mathscr{L}_{BC} = \lambda_1 MSE_{\Omega^*} + \lambda_2 \left(MSE_{\Gamma_1^*} + MSE_{\Gamma_2^*} + MSE_{\Gamma_3^*}\right) \tag{20}$$

where

$$MSE_{\Omega^*} = \frac{1}{N_{\Omega^*}} \sum_{i=1}^{N_{\Omega^*}} \| \nabla^{*4} \widehat{w^*}\left(\mathbf{x}_i^*; \mathbf{\Phi}\right) - \frac{q\left(\mathbf{x}_i^*\right)}{q_0} \|^2 \tag{21}$$

$$MSE_{\Gamma_1^*} = \frac{1}{N_{\Gamma_1^*}} \sum_{j=1}^{N_{\Gamma_1^*}} \left(\| \widehat{w^*}\left(\mathbf{x}_j^*; \mathbf{\Phi}\right) \|^2 + \| \frac{\partial \widehat{w^*}\left(\mathbf{x}_j^*; \mathbf{\Phi}\right)}{\partial n^*} \|^2 \right) \tag{22}$$

$$MSE_{\Gamma_2^*} = \frac{1}{N_{\Gamma_2^*}} \sum_{k=1}^{N_{\Gamma_2^*}} \left(\| \widehat{w^*}\left(\mathbf{x}_k^*; \mathbf{\Phi}\right) \|^2 + \| M_n^*\left(\mathbf{x}_k^*; \mathbf{\Phi}\right) - \widetilde{M}_n^* \|^2 \right) \tag{23}$$

Algorithm 1
PINNs for solving Kirchhoff plate bending problems.

Input: Physical parameters: D, E, ν, L, q_0; Calculate characteristic scale: $W_0 = \frac{q_0 L^4}{D}$; Number of collocation points for the domain N_Ω and boundaries $N_{\Gamma_1}, N_{\Gamma_2}, N_{\Gamma_3}$; loss weights λ_1, λ_2; learning rate α; number of training iterations T.

Initialize: A PINN neural network $\widehat{w^*}\left((x^*, y^*); \mathbf{\Phi}\right)$ with parameters $\mathbf{\Phi}_0$ (weights).

Data Preprocessing: Transform sampled coordinate physical values (x, y) to nondimensional coordinates $(x^*, y^*) = (x/L, y/L)$.

for $t = 0,1,2,\ldots,T-1$ **do**

Sample a batch of collocation points:

$$\{(x_i^*, y_i^*)\}_{i=1}^{N_{\Omega^*}} \subset \Omega^*$$

$$\{(x_j^*, y_j^*)_{\Gamma_1^*}\}_{j=1}^{N_{\Gamma_1^*}} \subset \Gamma_1^*$$

$$\{(x_k^*, y_k^*)_{\Gamma_2^*}\}_{k=1}^{N_{\Gamma_2^*}} \subset \Gamma_2^*$$

$$\{(x_l^*, y_l^*)_{\Gamma_3^*}\}_{l=1}^{N_{\Gamma_3^*}} \subset \Gamma_3^*$$

Compute the loss function $C(\mathbf{\Phi}) = \lambda_1 MSE_{\Omega^*} + \lambda_2\left(MSE_{\Gamma_1^*} + MSE_{\Gamma_2^*} + MSE_{\Gamma_3^*}\right)$ by evaluating the mean squared errors as defined in Equations (21)-(24) using the network outputs and their derivatives obtained via automatic differentiation.

Compute the gradient of the loss function with respect to the network parameters: $\nabla_{\mathbf{\Phi}} C(\mathbf{\Phi}_t)$.

Update the network parameters using a gradient descent-based optimizer (e.g., Adam):

$$\mathbf{\Phi}_{t+1} \leftarrow \mathbf{\Phi}_t - \alpha \nabla_{\mathbf{\Phi}} C(\mathbf{\Phi}_t)$$

end

Output: The optimized network parameters $\mathbf{\Phi}^* = \mathbf{\Phi}_T$, which define the PINN Solution $\widehat{w^*}\left((x^*, y^*); \mathbf{\Phi}\right)$.

Post-processing: Restore the prediction results to physical quantities: $w_{pred} = \widehat{w^*} \cdot W_0$.

$$MSE_{\Gamma_3^*} = \frac{1}{N_{\Gamma_3^*}} \sum_{l=1}^{N_{\Gamma_3^*}} \left(\| M_n^*(\mathbf{x}_l^*; \mathbf{\Phi}) \|^2 + \| V_n^*(\mathbf{x}_l^*; \mathbf{\Phi}) - \widetilde{V}_n^* \|^2 \right) \tag{24}$$

3.5. Optimization algorithm

The optimization objective of the neural network is to find suitable parameters of the neural network $\mathbf{\Phi}^*$ such that:

$$\mathbf{\Phi}^* = \underset{\phi}{\operatorname{argmin}} C(\mathbf{\Phi}) \tag{25}$$

Current optimization algorithms are mostly based on the basic idea of gradient descent, which can be expressed as:

$$\mathbf{\Phi}_{t+1} \leftarrow \mathbf{\Phi}_t - \alpha \frac{\partial C}{\partial \mathbf{\Phi}} \tag{26}$$

where $\mathbf{\Phi}_t$ represents the matrix of neural network parameters (weights and biases) in the $t-th$ iteration, α is the learning rate, and the operator ← denotes updating the value on the left with the value on the right. Common optimization algorithms based on gradient descent include Batch Gradient Descent (BGD), Stochastic Gradient Descent (SGD), Mini-batch Gradient Descent (MBGD), Momentum, Adaptive Gradient Algorithm (Adagrad) and Adam, among others [42]. The focus of this study is not on evaluating optimization algorithms. Therefore, the most commonly used Adam optimizer was employed for training across all case studies.

3.6. Algorithmic process

The algorithmic process of training the Physics-Informed Neural Network (PINN) to solve the Kirchhoff plate bending problem is summarized in Algorithm 1. The core idea is to iteratively minimize a composite loss function, which ensures that the network's output, $\widehat{w^*}((x^*, y^*); \mathbf{\Phi})$, simultaneously satisfies the governing partial differential equation within the domain and the specified conditions on the boundaries.

4. Numerical results and analysis

This section analyzes four numerical examples to validate the effectiveness and accuracy of the proposed method. All models in this study were implemented using the PyTorch deep learning framework,

Table 1
Hyperparameter Configuration for the Tanh-B6 KAN-based PINN Network Architecture.

Hyperparameter	Value	Description
Layers hidden	[2, 5, 5, 1]	Define the network hierarchical structure
Grid size	10	Number of B-spline meshes
Spline order	6	Degree of B-spline Polynomial
Base activation	nn.Tanh	Fixed Activation Function for Basic Components
Grid range	[−1, 1]	Initial range of the spline mesh
Grid eps	0.02	Weight of adaptive grid during grid update
Scale noise	0.1	Noise scale for spline weight initialization
Scale base	1.0	Scale of Initialization for Base Weights
Scale spline	1.0	Scale of Spline Weight Initialization

with accelerated computation performed on an NVIDIA RTX 4060 GPU. To rigorously evaluate whether the Tanh activation function is the key factor influencing the performance improvement of KAN neural networks, or whether the sixth-order B-spline curve is the crucial factor, we compare three distinct network architectures in each numerical example:

(1) MLP-PINN: The conventional Physics-Informed Neural Network built upon Multi-Layer Perceptrons (MLP), utilizing the Tanh activation function.
(2) B6 KAN-based PINN: A standard KAN-based PINN architecture employing sixth-order B-splines. This model serves as a baseline to isolate the specific influence of the activation function.
(3) Tanh-B6 KAN-based PINN: The proposed KAN-based PINN architecture, which integrates a learnable Tanh activation function with sixth-order B-splines.

The key hyperparameter configurations for the Tanh-B6 KAN-based architectures are detailed in Table 1. For the B6 KAN-based PINN model, the configuration is identical to that of the Tanh-B6 KAN-based PINN model, differing only in the choice of the base activation function. For fair comparison, the number of training iterations for all three models is set to 100,000, and the number of randomly generated boundary points and domain points in each training cycle is kept consistent. The width and depth of the MLP-PINN neural network were appropriately increased according to the complexity of the problem [43,44]. However, to validate the robustness of the proposed method, the neural network architecture for both KAN-based models (B6 and Tanh-B6) was fixed to [2, 5, 5, 1] across all examples.

To mitigate numerical instability arising from significant disparities in physical magnitudes and to accelerate network convergence, a non-dimensionalization strategy was employed across all numerical examples in this study. Specifically, the coordinates of the computational domain (x, y) were mapped to dimensionless coordinates (x^*, y^*), while the deflection w and bending moment M_n were scaled according to the dimensionless formulations derived in Section 2. Consequently, both the inputs and outputs of the neural networks consist of dimensionless quantities. Upon the completion of training, the predicted values were recovered to their physical dimensional forms via inverse transformation to facilitate a direct comparison with the theoretical solutions.

When formulating the loss function for a Physics-Informed Neural Network (PINN), the allocation of weights between the Partial Differential Equation (PDE) residual loss and the boundary condition (BC) loss is a critical consideration. In practice, a significant magnitude disparity often exists between the backpropagated gradients of these two loss components. This can lead to an imbalanced gradient flow, which impedes model convergence or results in suboptimal solutions. To address this, an effective manual weighting strategy involves assigning a substantially larger weight to the BC loss relative to the PDE loss [45]. This allocation is determined based on the initial magnitudes of the PDE and BC losses at the onset of training. The objective is to ensure that the weighted BC loss is approximately two orders of magnitude greater than the weighted PDE loss during the initial training phase.

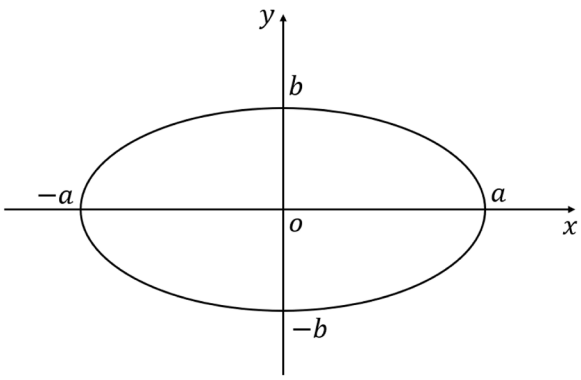


Fig. 3. Elliptical thin plate.

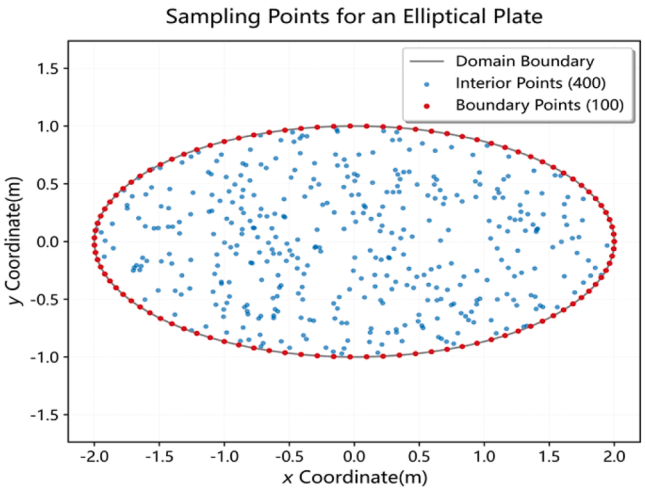


Fig. 4. Elliptical Thin Plate Sampling Diagram.

This heuristic is designed to guide the model to prioritize the satisfaction of boundary conditions during the initial epochs, before subsequently minimizing the governing PDE residual within the domain. The underlying rationale is to first establish a structurally correct solution framework that adheres to the boundary constraints, and thereafter resolve the finer details of the solution in the interior domain. As the network learns to accurately satisfy the boundary conditions, its output function tends to exhibit greater smoothness and regularity. This property is crucial for yielding stable and physically meaningful gradients, which are essential for the computation of the PDE-based loss component. Once the boundary constraints are sufficiently enforced, the training process can then progressively guide the model toward satisfying the internal physics governed by the PDE.

4.1. Bending problem of an elliptical thin plate clamped along the boundary and subjected to uniformly distributed load

Consider an elliptical thin plate (Fig. 3) clamped along the boundary, with semi-major axis $a = 2m$, semi-minor axis $b = 1m$, thickness $h = 0.1m$, Poisson's ratio $\nu = 0.3$, and elastic modulus $E = 10920Pa$. The plate is subjected to a transversely uniformly distributed load $q_0 = 1N/m^2$. In this numerical calculation, we adopt a characteristic length of $L = a = 2m$ for non-dimensionalization, while other physical quantities are scaled according to the dimensionless formulas derived in Section 2. The boundary condition equation for the thin plate is Eq. (4). The exact theoretical solution for this problem is:

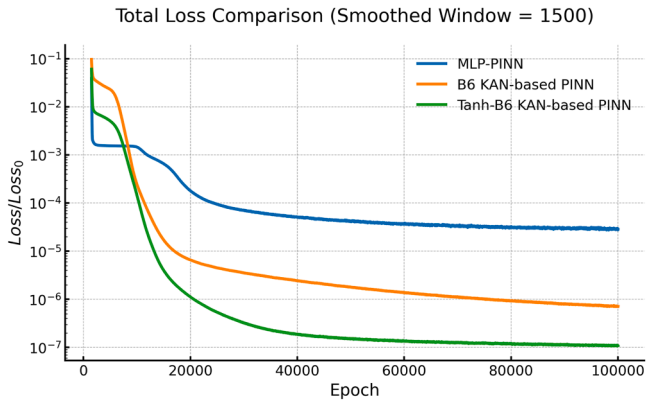


Fig. 5. Total loss curve of elliptical thin plate.

$$w = \frac{q_0 a^4 b^4 \left(1 - \frac{x^2}{a^2} - \frac{y^2}{b^2}\right)^2}{8D\left(3b^4 + 2a^2b^2 + 3a^4\right)} \tag{27}$$

where D is the flexural rigidity of the thin plate.

In this numerical example, 300 domain collocation points and 100 boundary points were randomly sampled within the computational domain and on its boundaries, respectively, during each training epoch (as illustrated in Fig. 4). The MLP-PINN was constructed using a [2, 20, 20, 20, 20, 20, 1] architecture, resulting in a total of 2181 trainable parameters. In contrast, the B6 KAN-based PINN and the Tanh-B6 KAN-based PINN employed a [2, 5, 5, 1] architecture, comprising only 51 parameters. All models were optimized using an identical total loss function, defined as $Total_{loss} = PDE_{loss} + 10^4 \times BC_{loss}$, with a learning rate of 1×10^{-4}. The loss convergence curves for the training process are presented in Fig. 5. The results provide a clear hierarchy of performance. The total loss of the MLP-PINN converged to the 10^{-5} magnitude after 100,000 iterations. In contrast, the B6 KAN-based PINN demonstrated significantly superior convergence, breaking through the bottleneck encountered by the MLP and reaching a loss magnitude of 10^{-7}. This confirms that the C^4 continuity provided by the sixth-order B-splines is the primary factor in resolving the high-order derivative difficulties. However, the Tanh-B6 KAN-based PINN exhibited the best performance, achieving a 10^{-7} loss magnitude in 20,000 iterations and eventually converging to nearly 10^{-8} magnitude. This indicates that while the high-order splines are fundamental, the learnable Tanh activation function further accelerates convergence and deepens the loss minimum.

Fig. 6 illustrates the two-dimensional deflection distributions obtained from the MLP-PINN, the B6 KAN-based PINN, the Tanh-B6 KAN-based PINN, and the exact theoretical solution. To facilitate a detailed comparison, Fig. 7 presents the numerical solution profiles along the $x = 0$ and $y = 0$ axes, as well as the solution profile along the entire boundary. These visualizations, complemented by the absolute error distributions shown in Fig. 8, unequivocally demonstrate that both KAN-based PINN approaches significantly outperform the MLP-PINN. This conclusion is quantitatively corroborated by the statistical results summarized in Tables 2 and 3. In this numerical case, KAN-based PINN approaches yield both a relative L_2 norm error and a mean absolute error (MAE) within the domain that are three orders of magnitude lower than those of the MLP-PINN. Furthermore, their MAE along the entire boundary (relative to the theoretical value of 0) is similarly reduced by three orders of magnitude. Compared to the B6 KAN-based PINN, the Tanh-B6 KAN-based PINN further reduces the relative L_2 norm error by 0.026 %, demonstrating that the global support characteristics of the Tanh activation effectively complement the local approximation capabilities of the B-splines.

It is worth noting that neither of the KAN-based PINN models strictly yields an exact zero value at the boundaries. This behavior is an inherent characteristic of the PINN methodology, which incorporates the boundary error (or residual) into the total training loss function. During

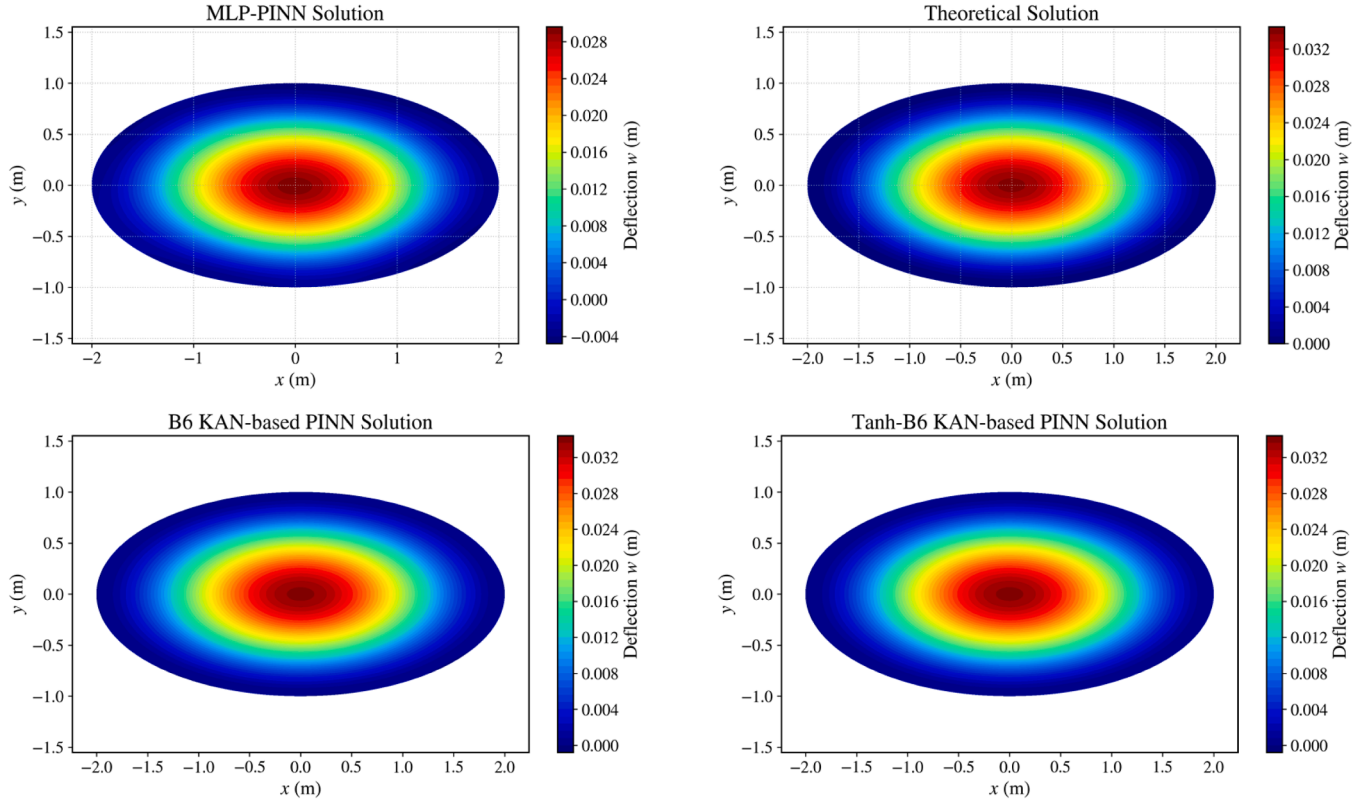
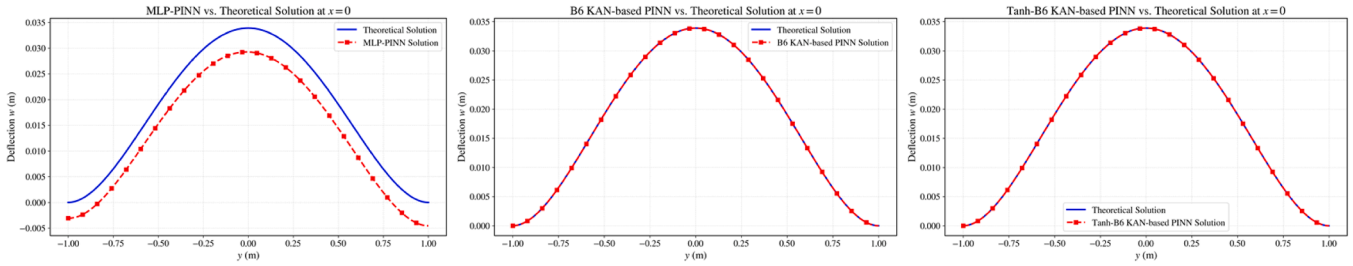
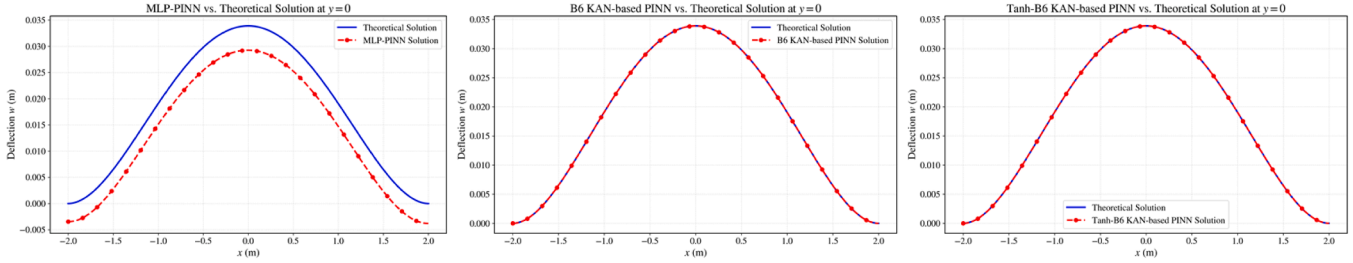


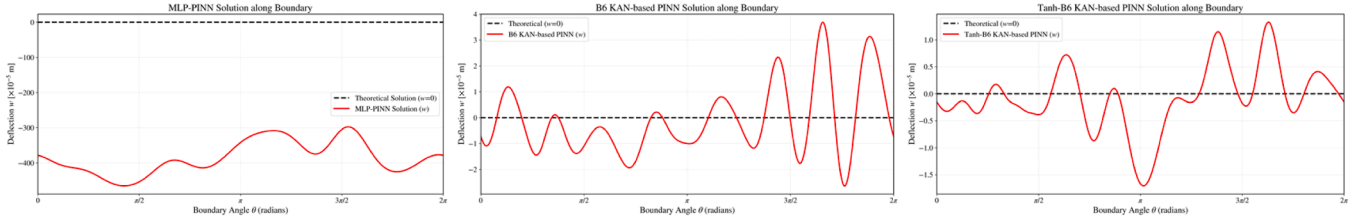
Fig. 6. 2D solutions of elliptical thin plate.



(a) Solutions along the axis $x = 0$.



(b) Solutions along the axis $y = 0$.



(c) Solutions along boundary.

Fig. 7. Numerical curves of elliptical thin plate solutions.

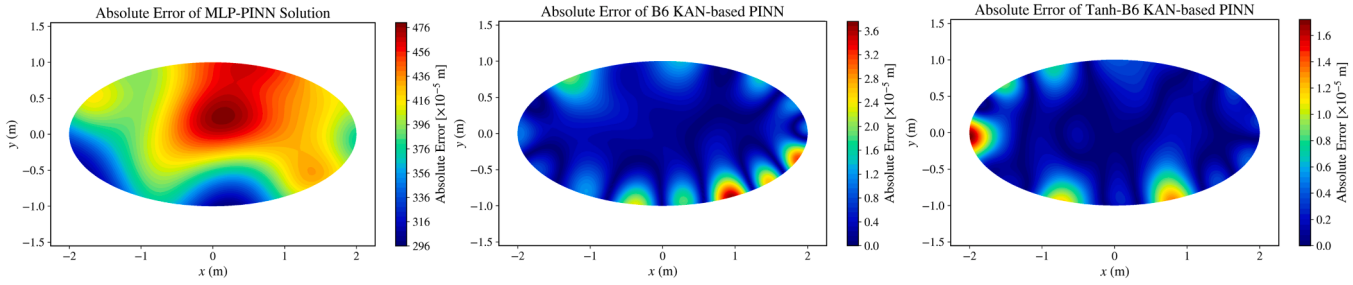


Fig. 8. Absolute Error between PINNs solutions and Theoretical solution for elliptical thin plate.

Table 2
Error Statistics for the Bending Problem of elliptical Thin Plate.

Error	MLP-PINN	B6 KAN-based PINN	Tanh-B6 KAN-based PINN
Relative L_2 Norm Error	19.7237%	0.0381 %	0.0121%
Mean Absolute Error ($\times 10^{-5}m$)	425.74	0.4231	0.1593
Max Absolute Error ($\times 10^{-5}m$)	479.85	3.6942	1.7124

Table 3
Error Statistics at the Boundary of elliptical Thin Plate.

Error ($\times 10^{-5}m$)	MLP-PINN	B6 KAN-based PINN	Tanh-B6 KAN-based PINN
Mean Absolute Error	358.49	1.0591	0.4360
Root Mean Squared Error	388.21	1.3279	0.5986
Max Absolute Error	465.24	3.6889	1.7044

the optimization process, it is computationally impractical to drive this error component to exactly zero. Instead, the optimizer minimizes the loss, reducing the boundary error to a value arbitrarily close to zero. This

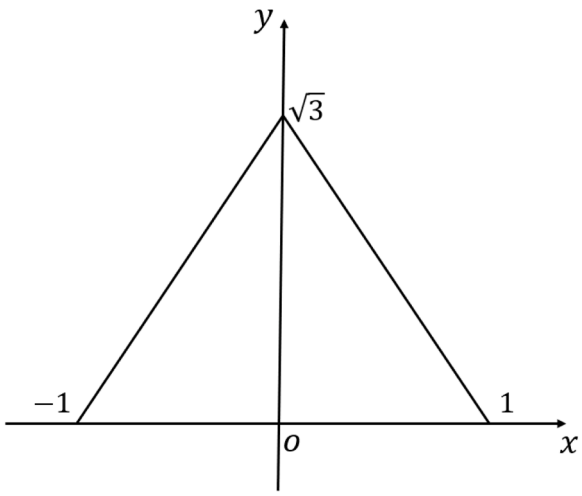


Fig. 9. Equilateral triangular thin plate.

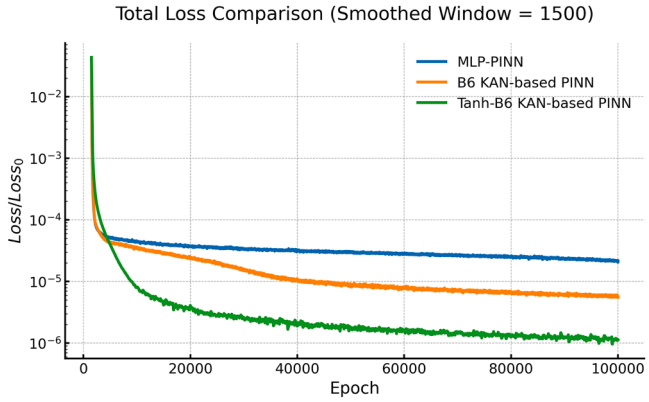


Fig. 10. Total loss curve of equilateral triangular thin plate.

near-zero residual is entirely acceptable for a numerical approximation algorithm.

4.2. Bending problem of an equilateral triangular thin plate simply supported along the boundary and subjected to uniformly distributed bending moment

Consider an equilateral triangular thin plate (Fig. 9) simply supported along the boundary, with side length $a = 2m$, height $b = \sqrt{3}m$, thickness $h = 0.1m$, Poisson's ratio $v = 0.3$, and elastic modulus $E = 10920Pa$. A uniformly distributed bending moment $M_n = 1N \cdot m$ acts along the boundary. In this numerical calculation, we adopt a characteristic length of $L = a = 2m$ for non-dimensionalization, while other physical quantities are scaled according to the dimensionless formulas derived in Section 2. The boundary condition equation for the thin plate is Eq. (5). The exact theoretical solution for this problem is:

$$w = \frac{M_n}{4bD}\left(y^3 - 3x^2y + 2bx^2 - \frac{b^2y}{3} + \frac{2b^3}{9}\right) \tag{28}$$

where D is the flexural rigidity of the thin plate.

For this numerical validation, 180 boundary points and 340 domain points were randomly sampled within each training epoch. We compare three distinct network architectures: a conventional MLP-PINN, a B6 KAN-based PINN, and a Tanh-B6 KAN-based PINN. The MLP-PINN utilized a [2, 20, 20, 20, 20, 1] architecture, comprising 1761 trainable parameters. In contrast, the B6 KAN-based PINN and the Tanh-B6 KAN-based PINN employed a [2, 5, 5, 1] architecture, comprising only 51 parameters. All models were trained using an identical learning rate of 1×10^{-3} and a composite loss function, weighted as $Total_{loss} = PDE_{loss} + 10^3 \times BC_{loss}$. The comparative training histories are depicted in Fig. 10. The results clearly demonstrate a tiered improvement in convergence performance. The MLP-PINN exhibits the slowest convergence and highest final loss. The B6 KAN-based PINN achieves significantly more effective convergence than the MLP-PINN. This substantial improvement with 30 times fewer parameters confirms the efficiency of high-order B-splines in approximating the solution. However, the Tanh-B6 KAN-based PINN further surpasses the B6 baseline, achieving the fastest convergence speed and the lowest final loss. Notably, beyond their superior convergence performance, both KAN-based approaches facilitate a parameter count reduction of two orders of magnitude while offering substantially enhanced model interpretability.

Fig. 11 illustrates the 2D deflection distributions for the MLP-PINN, the B6 KAN-based PINN, the Tanh-B6 KAN-based PINN, and the exact theoretical solutions. For a detailed comparison, Fig. 12 presents the numerical curves of the models against the theoretical solution at the $x = 0$ cross-section and against the theoretical value of 0 along the entire boundary. Combined with the absolute error distributions depicted in Fig. 13, these results clearly demonstrate that the solution accuracy of the Tanh-B6 KAN-based PINN is superior to both comparative models. Furthermore, the quantitative statistics in Tables 4 and 5 provide robust support for this conclusion. In this numerical case, the relative L_2 norm error and the mean absolute error (MAE) of the Tanh-B6 KAN-based PINN within the domain are both two orders of magnitude lower than those of the MLP-PINN. Crucially, when compared to the B6 KAN-based PINN, the Tanh-B6 KAN-based PINN further reduces the relative L_2 norm error by 1.0302 %. This additional gain suggests that while the sixth-order splines capture the primary physics dictated by the fourth-order PDE, the inclusion of the global Tanh activation helps to better approximate the uniform bending moment distribution and minimize residuals across the domain. Moreover, its mean absolute error along the entire boundary is reduced by two orders of magnitude compared to the MLP-PINN, and shows a distinct improvement over the B6 KAN-based baseline.

4.3. Bending problem of a rectangular thin plate simply supported along the boundary and subjected to sinusoidal distributed load

Consider a rectangular thin plate simply supported around the boundary, with side lengths $a = 1m$, $b = 1m$, thickness $h = 0.1m$, Poisson's ratio $v = 0.3$, and elastic modulus $E = 10920Pa$. The plate is subjected to a transverse nonlinear distributed load $q(x, y) = q_0 \sin(\pi x)\sin(\pi y)$, where $q_0 = 1N/m^2$ (Fig. 14). In this numerical calculation, we adopt a characteristic length of $L = a = b = 1m$ for non-dimensionalization, while other physical quantities are scaled according to the dimensionless formulas derived in Section 2. The boundary condition equation for the thin plate is Eq. (5).

In the context of classical plate theory, the bending problem of a rectangular thin plate under a transverse load $q(x, y)$, with all four edges simply supported (SSSS), is amenable to an analytical solution via the Navier method. This approach is predicated on representing the plate's deflection, $w(x, y)$, as a double trigonometric (Fourier) series, which inherently satisfies the boundary conditions. The applied load $q(x, y)$ is also expanded in a similar series. By utilizing the orthogonality of the trigonometric basis functions, the unknown coefficients of the deflection series can be determined directly. For the specific case of a single sinusoidal load component, $q(x,y) = q_0 \sin(\pi x)\sin(\pi y)$, where a and b are the plate dimensions, the Navier solution provides the exact analytical expression for the deflection:

$$w(x,y) = \frac{q_0}{D\pi^4\left(\frac{1}{a^2} + \frac{1}{b^2}\right)^2}\sin\left(\frac{\pi x}{a}\right)\sin\left(\frac{\pi y}{b}\right) \tag{29}$$

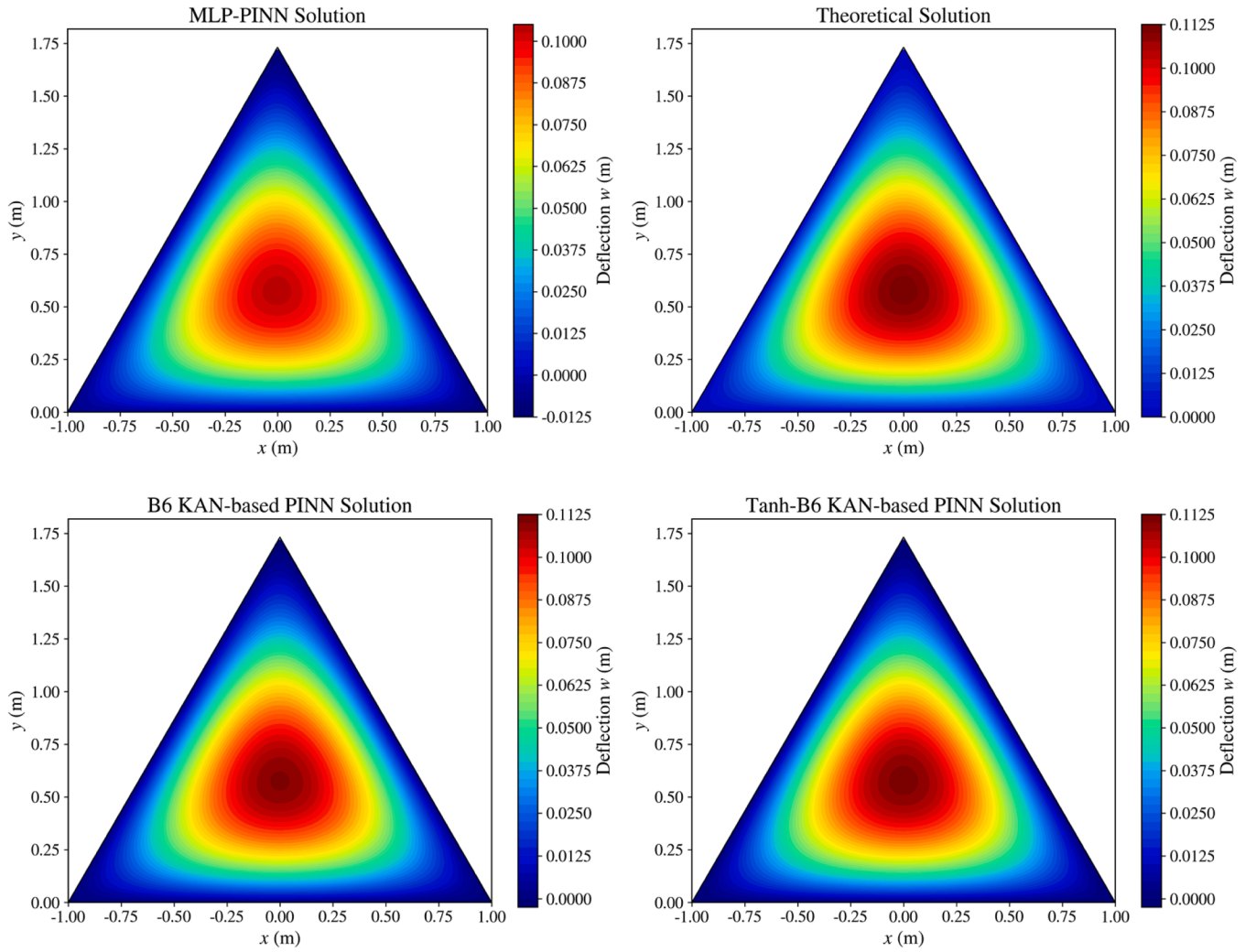
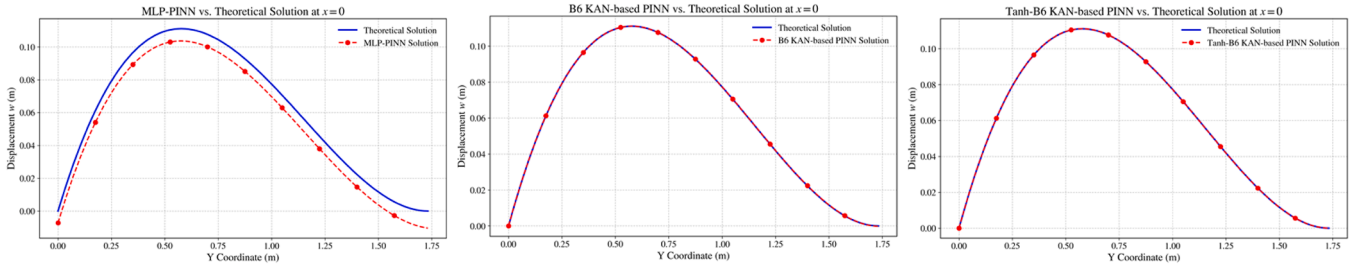


Fig. 11. 2D solutions of equilateral triangular thin plate.

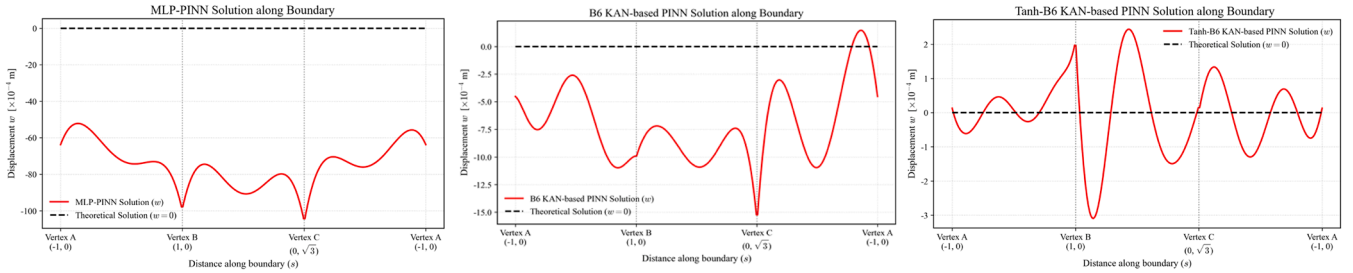
where D is the flexural rigidity of the thin plate.

In this numerical example, 120 boundary points and 400 domain collocation points were randomly sampled during each training epoch. The MLP-PINN was trained using a [2, 40, 40, 40, 40, 40, 40, 40, 1] network architecture, totaling 10,001 parameters. In contrast, the B6 KAN-based PINN and the Tanh-B6 KAN-based PINN employed a [2, 5, 5, 1] architecture, comprising only 51 parameters. All models were optimized using an identical total loss function, defined as $Total_{loss} = PDE_{loss} + 10^4 \times BC_{loss}$, with a learning rate of 1×10^{-3}. The loss histories, presented in Fig. 15, reveal distinct training dynamics. The MLP-PINN exhibits significantly larger oscillations and slower convergence. This instability is attributed to multi-objective optimization conflicts [46, 47], which arise because the loss function for a high-order PDE must simultaneously optimize multiple competing terms, such as the equation residual and various boundary conditions. These loss terms often vary significantly in magnitude; specifically, residuals from high-order derivatives tend to be substantially larger than those from low-order terms. In this specific case, the loss function incorporates zero-order derivatives (corresponding to displacement constraints), second-order derivatives (corresponding to bending moment constraints), and fourth-order derivatives (corresponding to the governing equation). These distinct derivative orders compete during backpropagation. The global parameterization of the MLP struggles to balance these competing terms. In contrast, the B6 KAN-based PINN significantly mitigates these numerical oscillations. The intrinsic smoothness and local support of the sixth-order B-splines allow the network to resolve high-order derivatives without the gradient instability that plagues the MLP, resulting in a much smoother loss curve and a final loss magnitude of 10^{-7}. However, the Tanh-B6 KAN-based PINN achieves superior performance. By integrating the Tanh activation, which naturally captures global low-frequency patterns, with the local high-frequency adjustment capability of B-splines, the Tanh-B6 model achieves both a faster convergence rate and a lower final loss floor than the B6 baseline. Notably, the Tanh-B6 KAN-based PINN accomplishes this with a parameter count that is three orders of magnitude smaller than the MLP-PINN.

Fig. 16 illustrates the 2D deflection distributions for the MLP-PINN, the B6 KAN-based PINN, the Tanh-B6 KAN-based PINN, and the exact Navier solutions. For a detailed comparison, Fig. 17 presents the numerical curves of the models against the Navier solution at the $y = 0.5$ cross-section and against the theoretical value of 0 along the entire boundary. Combined with the absolute error distributions depicted in Fig. 18, these results clearly demonstrate the performance hierarchy. The quantitative statistics in Tables 6 and 7 provide robust support for this conclusion. In this numerical case, the relative L_2 norm error and the mean absolute error of the Tanh-B6 KAN-based PINN within the domain are both one order of magnitude lower than those of the MLP-PINN. Furthermore, compared to the Standard B6 KAN-based PINN, the Tanh-B6 architecture reduces the relative L_2 norm error by 2.4726 %. This indicates that although the sixth-order spline is crucial for stability, the addition of the Tanh activation function in the Tanh-B6 KAN-based



(a) Solutions along the axis $x = 0$.



(b) Solutions along boundary.

Fig. 12. Numerical curves of equilateral triangular thin plate solutions.

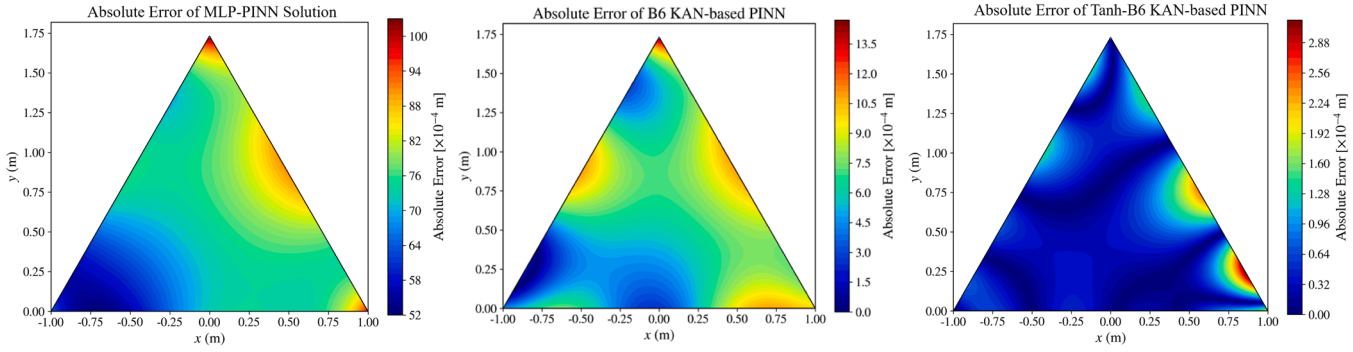


Fig. 13. Absolute Error between PINNs solutions and Theoretical solution for equilateral triangular thin plate.

Table 4
Error Statistics for the Bending Problem of Equilateral Triangular Thin Plate.

Error	MLP-PINN	B6 KAN-based PINN	Tanh-B6 KAN-based PINN
Relative L_2 Norm Error	12.3644%	1.1378 %	0.1076%
Mean Absolute Error ($\times 10^{-4}m$)	73.434	6.4863	0.4724
Max Absolute Error ($\times 10^{-4}m$)	102.51	14.4651	3.1025

Table 5
Error Statistics at the Boundary of Equilateral Triangular Thin Plate.

Error ($\times 10^{-4}m$)	MLP-PINN	B6 KAN-based PINN	Tanh-B6 KAN-based PINN
Mean Absolute Error	74.534	7.1846	0.8468
Root Mean Squared Error	75.430	7.8541	1.1036
Max Absolute Error	104.39	15.2373	3.0953

PINN architecture further stabilizes the training process. This is because the Tanh activation function helps to more effectively capture the low-frequency components of the solution, enabling the B-spline to focus on fine-tuning the local high-order variations. Moreover, the mean absolute error of the Tanh-B6 KAN-based PINN along the entire boundary is reduced by one order of magnitude compared to the MLP-PINN.

4.4. Bending problem of an L-shaped thin plate simply supported along the boundary and subjected to uniformly distributed load

Consider an L-shaped thin plate (Fig. 19) simply supported around its boundary, with thickness $h = 0.1m$, Poisson's ratio $\nu = 0.3$, and elastic modulus $E = 10920Pa$. The plate is subjected to a uniformly distributed transverse load $q_0 = 1N/m^2$. In this numerical calculation, we adopt a characteristic length of $L = 1m$ for non-dimensionalization, while other physical quantities are scaled according to the dimensionless formulas derived in Section 2. The boundary condition equation for the thin plate is Eq. (5).

For an L-shaped thin plate, due to its composition of three orthogonal rectangular regions, its boundary contains multiple right-angle turns, particularly 270° interior angles. This non-convex polygonal geometry

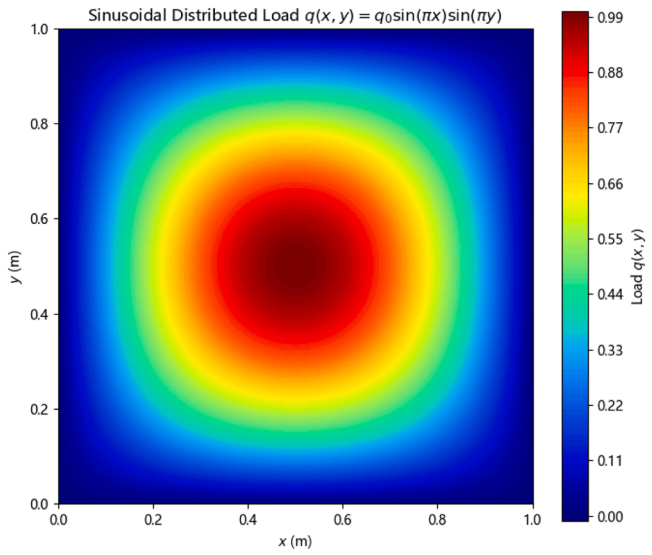


Fig. 14. Rectangular thin plate subjected to sinusoidal distributed load.

disrupts the symmetry of the Cartesian coordinate system: it is neither able to maintain translational symmetry in a single direction (such as the periodic boundary required by Fourier series) nor cover the entire area with radial symmetry using polar coordinates. Therefore, this numerical example compares the prediction results of the PINNs with the results of Abaqus finite element to verify the accuracy.

In this numerical example, 320 boundary points and 800 domain points were randomly sampled during each training iteration, and 100 additional dense sampling points were added within a circle with a radius of $0.2m$ around the inner corner point (0.5, 0.5) (Fig. 20). The MLP-PINN was trained using a neural network architecture of [2, 40, 40, 40, 40, 40, 40, 40, 40, 1], with a total of 11,641 parameters. In contrast, the B6 KAN-based PINN and the Tanh-B6 KAN-based PINN employed a [2, 5, 5, 1] architecture, comprising only 51 parameters. All models were optimized using an identical total loss function, defined as $Total_{loss} = PDE_{loss} + 10^6 \times BC_{loss}$, with a learning rate of 1×10^{-3}. The resulting loss curves, depicted in Fig. 21, demonstrate a clear performance hierarchy. The MLP-PINN exhibits slow convergence and large oscillation amplitudes. In contrast, the B6 KAN-based PINN demonstrates a marked improvement in stability and convergence speed, settling at a lower loss magnitude. However, the Tanh-B6 KAN-based PINN outperforms both comparative models, achieving the fastest convergence rate and the lowest final loss. As observed from the loss curves, the Tanh-B6 KAN-based PINN exhibits the smallest oscillation amplitudes, indicating that the hybrid Tanh-B-spline architecture provides the most stable training dynamics even in complex geometries. Furthermore, both KAN-based models achieve this superior performance with a parameter count that is three orders of magnitude smaller than that of the MLP-PINN.

In solving this problem using Abaqus, to evaluate mesh convergence, a series of progressively refined mesh sizes (100 mm, 75 mm, 50 mm, 25 mm, 10 mm, 7.5 mm and 5 mm) were used to discretize the model. The maximum deflection value was taken as the key indicator for assessing convergence. As shown in Fig. 22, when the mesh size was reduced to 5 mm, the change in the maximum deflection value stabilized, indicating that the numerical solution had converged and become independent of the mesh size. Therefore, the results calculated at this mesh scale can be considered to have achieved convergence. Consequently, the results obtained from the Abaqus mesh model with a 5 mm element size were used as the benchmark.

Fig. 23 shows the 2D deflection plots for the MLP-PINN, the B6 KAN-based PINN, the Tanh-B6 KAN-based PINN, and the Abaqus finite element results. From the 2D deflection comparison plots, it can be seen that all PINNs solutions exhibit some degree of non-convergence at the boundaries, particularly at the intersections between boundaries. According to the Sobolev trace theorem [48], the loss function's domain loss norm and boundary loss norm are not on the same order of magnitude. During backpropagation, this can easily lead to the boundary loss being "neglected". Even when the weight of the boundary loss is increased and dense sampling is performed at the inner corner point, the issue persists to varying degrees across all models. Our empirical results indicate that the boundary loss norm at the initial moment is typically 1 to 3 orders of magnitude lower than the domain loss norm.

As evidenced by the absolute error data benchmarked against the Abaqus FEM solution (Table 8), along with the boundary value curves (Fig. 24) and statistical metrics (Table 9), the Tanh-B6 KAN-based PINN solution demonstrates the highest overall accuracy among the three methods. Compared to the MLP-PINN, the Tanh-B6 KAN-based PINN achieves a reduction in maximum displacement error by two orders of magnitude and a reduction in the mean absolute boundary error by one order of magnitude. Furthermore, compared with the standard B6 KAN-based PINN, the proposed Tanh-B6 architecture achieves a reduction in both the maximum deflection error and the average absolute error at the boundary.

However, a detailed comparison of the maximum absolute errors on the L-shaped plate boundary (Table 9), particularly at the inner corner and adjacent points (Table 10), reveals a significant exception. Both KAN-based architectures (B6 and Tanh-B6) exhibit a higher absolute error at the inner (re-entrant) corner than the MLP-PINN. This finding suggests that the KAN-based PINNs do not offer a significant advantage over the MLP-PINN when addressing localized singularities in complex geometries. This limitation arises because the KAN architecture utilizes a spline grid that is rectangular in high dimensions, making it inherently better suited for regular geometric domains [37]. Nevertheless, it is worth noting that the Tanh-B6 KAN-based PINN still performs slightly better than the Standard B6 KAN-based PINN at the inner corner points, implying that the global support of the Tanh activation partially mitigates the limitations of the rectangular spline grid.

5. Investigation of Tanh-B6 KAN-based PINN network architecture parameters

5.1. Impact of the node bias term on network performance

To address the necessity of the bias term β mentioned in Eq. (9), we investigated its impact on the network's performance. Theoretically, the bias term aids in function fitting; however, the learnable B-spline basis functions used in the Tanh-B6 KAN inherently capture certain shift interpretations. In the Tanh-B6 KAN architecture proposed in this study, the explicit node bias term is omitted. This design choice is theoretically grounded in the mathematical properties of B-spline functions, specifically their Partition of Unity [40].

In KAN with the bias term, the output of the $j-th$ neuron in the $l-th$ layer is defined as (recalling Eq. (9)):

$$z_j^l = \sum_{k=1}^{n_{l-1}} \phi_{jk}^l \left(z_k^{l-1}\right) + \beta_j^l \tag{9()}$$

In our Tanh-B6 KAN, the activation function $\phi(x)$ consists of a Tanh component and a B-spline component (recalling Eq. (11)):

$$\phi(x) = a \cdot \mathrm{Tanh}(b \cdot x) + \sum_{i=1}^{G} c_i B_{i,6}(x) \tag{11()}$$

According to the Partition of Unity property of B-splines [40], for $k-th$ order B-spline basis functions (where $k = 6$ in this paper) defined on a knot vector $\mathbf{T} = \{t_0, t_1, \ldots, t_N\}$, the sum of all basis functions within the internal domain $[t_{k-1}, t_{N-k+1}]$ is identically equal to 1:

$$\sum_{i=1}^{G} B_{i,k}(x) \equiv 1, \forall x \in [t_{k-1}, t_{N-k+1}] \tag{30}$$

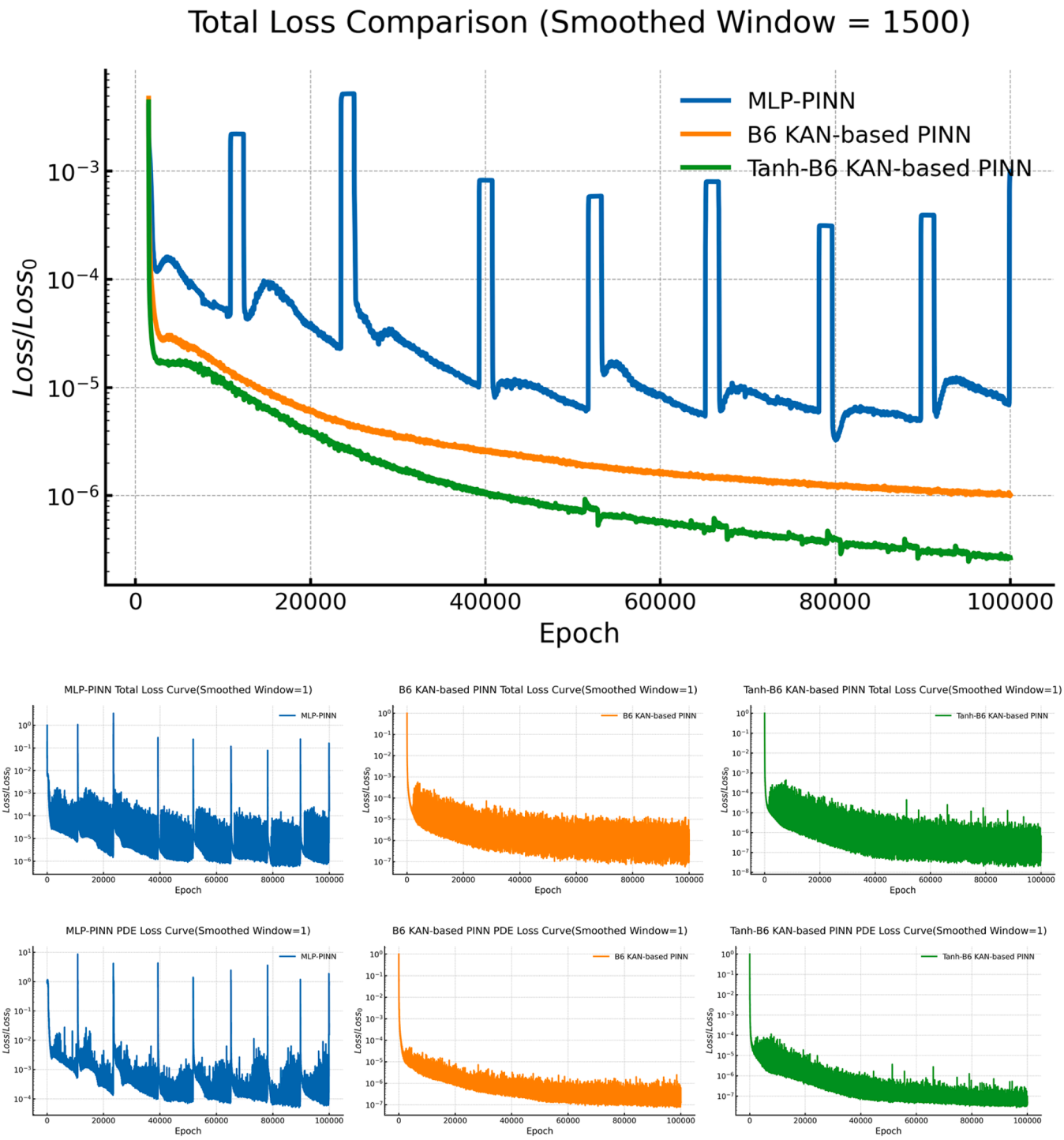


Fig. 15. PINNs loss curves for rectangular thin plate.

This implies that any constant bias β can be mathematically absorbed into the coefficients c_i of the B-splines. If a constant bias β is added to the activation function, the derivation of equivalence is as follows:

$$\begin{aligned}\phi(x)+\beta &= a\cdot Tanh(b\cdot x)+\sum_{i=1}^{G}c_i B_{i,6}(x)+\beta\cdot 1\\ &= a\cdot Tanh(b\cdot x)+\sum_{i=1}^{G}c_i B_{i,6}(x)+\beta\cdot\left(\sum_{i=1}^{G}B_{i,6}(x)\right)\\ &= a\cdot Tanh(b\cdot x)+\sum_{i=1}^{G}(c_i+\beta)B_{i,6}(x)\\ &= a\cdot Tanh(b\cdot x)+\sum_{i=1}^{G}c_i^{'}B_{i,6}(x)\end{aligned} \tag{31}$$

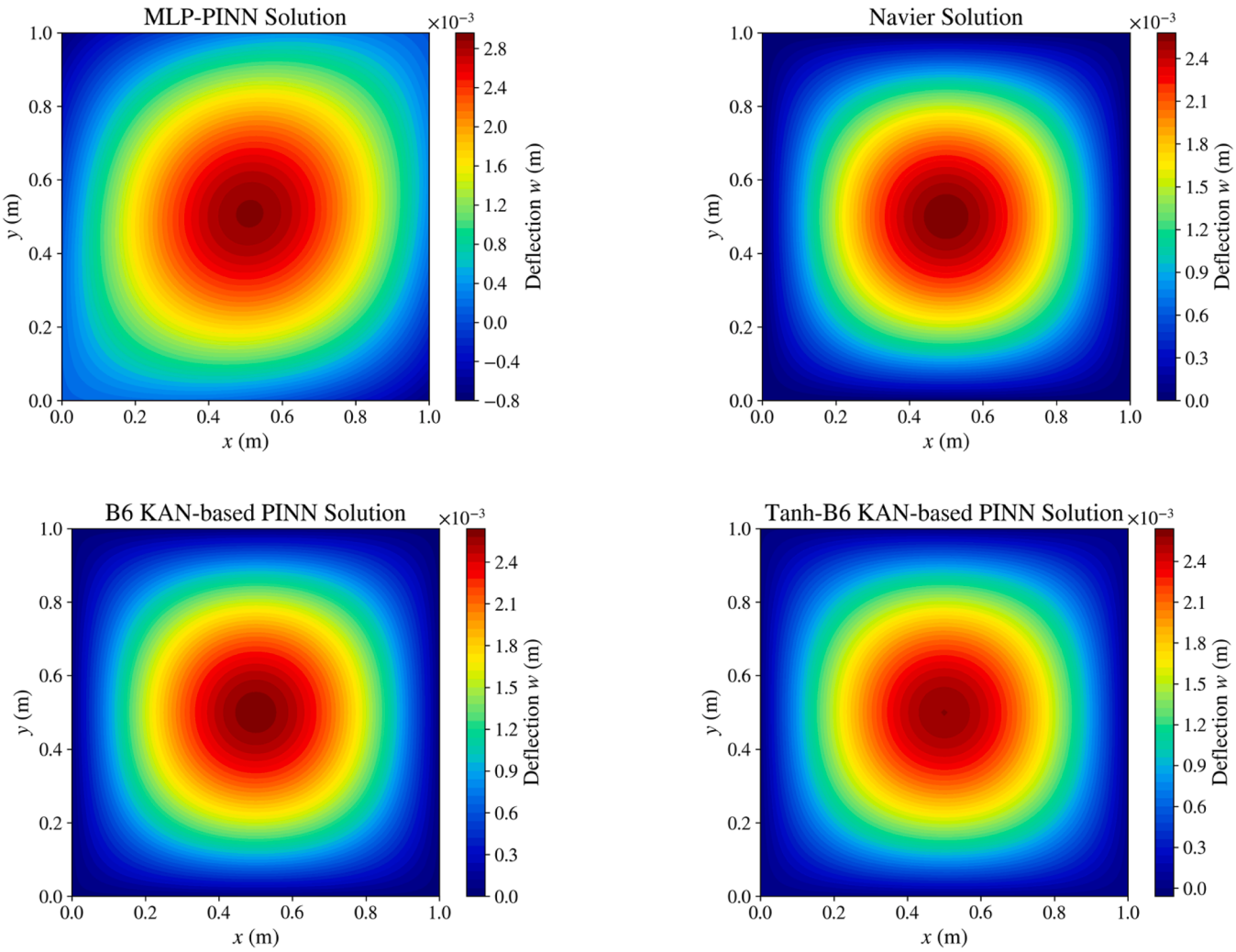
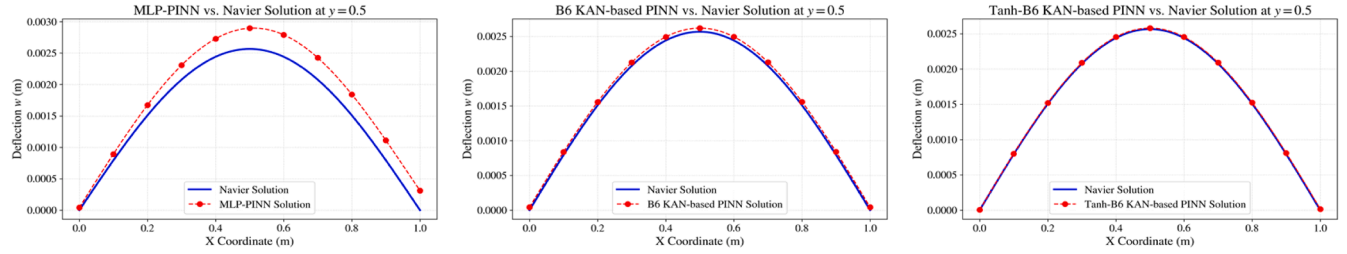
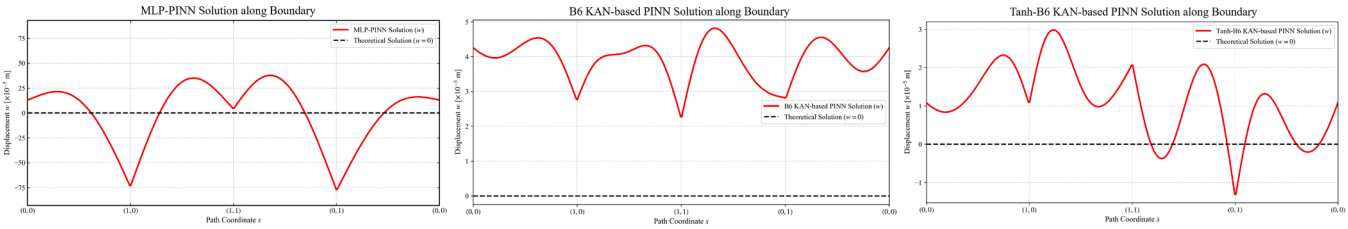


Fig. 16. 2D solutions of rectangular thin plate.



(a) Solutions along the axis $y = 0.5$.



(b) Solutions along boundary.

Fig. 17. Numerical curves of rectangular thin plate solutions.

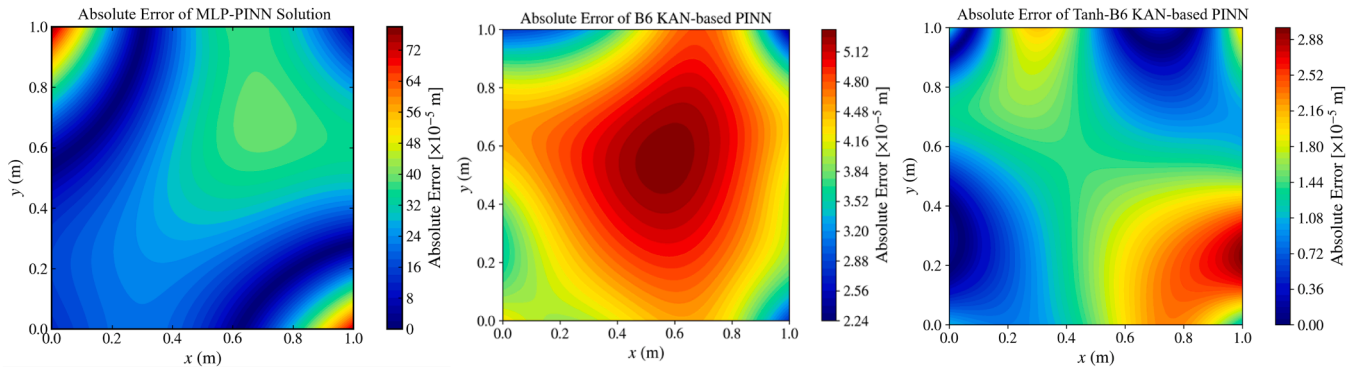


Fig. 18. Absolute Error between PINNs solutions and Theoretical solution for rectangular thin plate.

Table 6
Error Statistics for the Bending Problem of rectangular Thin Plate.

Error	MLP-PINN	B6 KAN-based PINN	Tanh-B6 KAN-based PINN
Relative L_2 Norm Error	20.6281%	3.6137 %	1.1411%
Mean Absolute Error ($\times 10^{-5}m$)	23.5940	4.5629	1.3095
Max Absolute Error ($\times 10^{-5}m$)	76.7714	5.3411	2.9853

Table 7
Error statistics at the boundary of rectangular thin plate.

Error ($\times 10^{-5}m$)	MLP-PINN	B6 KAN-based PINN	Tanh-B6 KAN-based PINN
Mean Absolute Error	25.3514	3.8936	1.2175
Root Mean Squared Error	30.6167	3.9321	1.4379
Max Absolute Error	76.7724	4.8138	2.9867

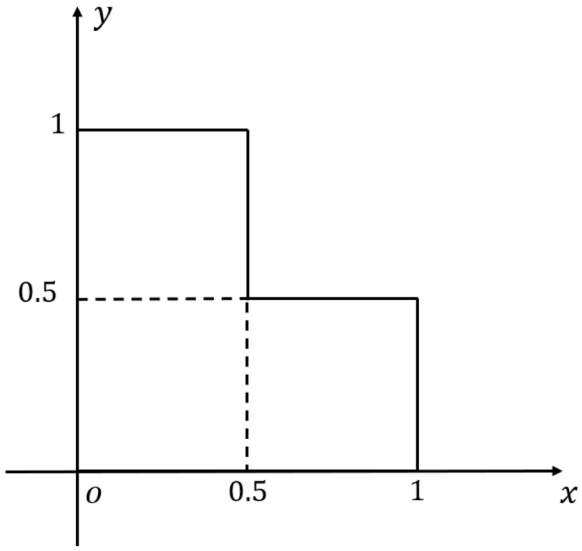


Fig. 19. L-shaped thin plate.

where $c_i^{'} = c_i + \beta$ represents the updated learnable coefficients.

Eq. (31) demonstrates that the explicit bias term β is linearly dependent on the B-spline coefficients c_i. During the optimization process, the neural network can implicitly learn the necessary bias β by adaptively adjusting the coefficients c_i.

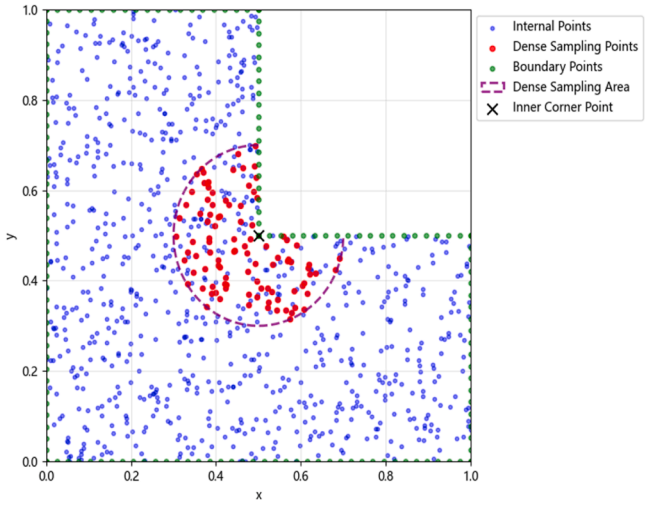


Fig. 20. L-shaped Thin Plate Sampling Diagram.

To validate this theoretical derivation in the context of Kirchhoff thin plate bending problems, we conducted an ablation study. Using the elliptical thin plate case (Section 4.1) as a benchmark, we compared the convergence performance of the architecture "Without Bias Term" (our proposed method) versus "With Bias Term " (explicitly adding learnable bias), while keeping all other hyperparameters fixed.

Fig. 25 illustrates the loss convergence curves for both architectures. It can be observed that the architecture without the bias term exhibits a faster convergence rate and achieves a lower final loss value compared to the architecture with the bias term. The inclusion of the bias term introduces additional optimization complexity, leading to slower convergence. This conclusion is further supported by the error analysis presented in Fig. 26 and the statistical metrics in Tables 11 and 12. The Tanh-B6 KAN-based PINN without the bias term outperforms the version with bias across all metrics, achieving lower relative L_2 norm errors and Mean Absolute Errors (MAE) both within the domain and along the boundaries.

These findings suggest that for the specific application of solving high-order PDEs using Tanh-B6 KAN, the flexible B-spline basis functions are sufficiently capable of capturing the necessary offsets. Consequently, the explicit bias term β_j^l is indeed unnecessary and, if included, may hinder the optimization efficiency of the Physics-Informed Neural Network. Therefore, removing the bias term is a valid and effective strategy for this study.

5.2. Impact of the number of neurons and the number of hidden layers

Taking the bending problem of an elliptical thin plate with clamped

a

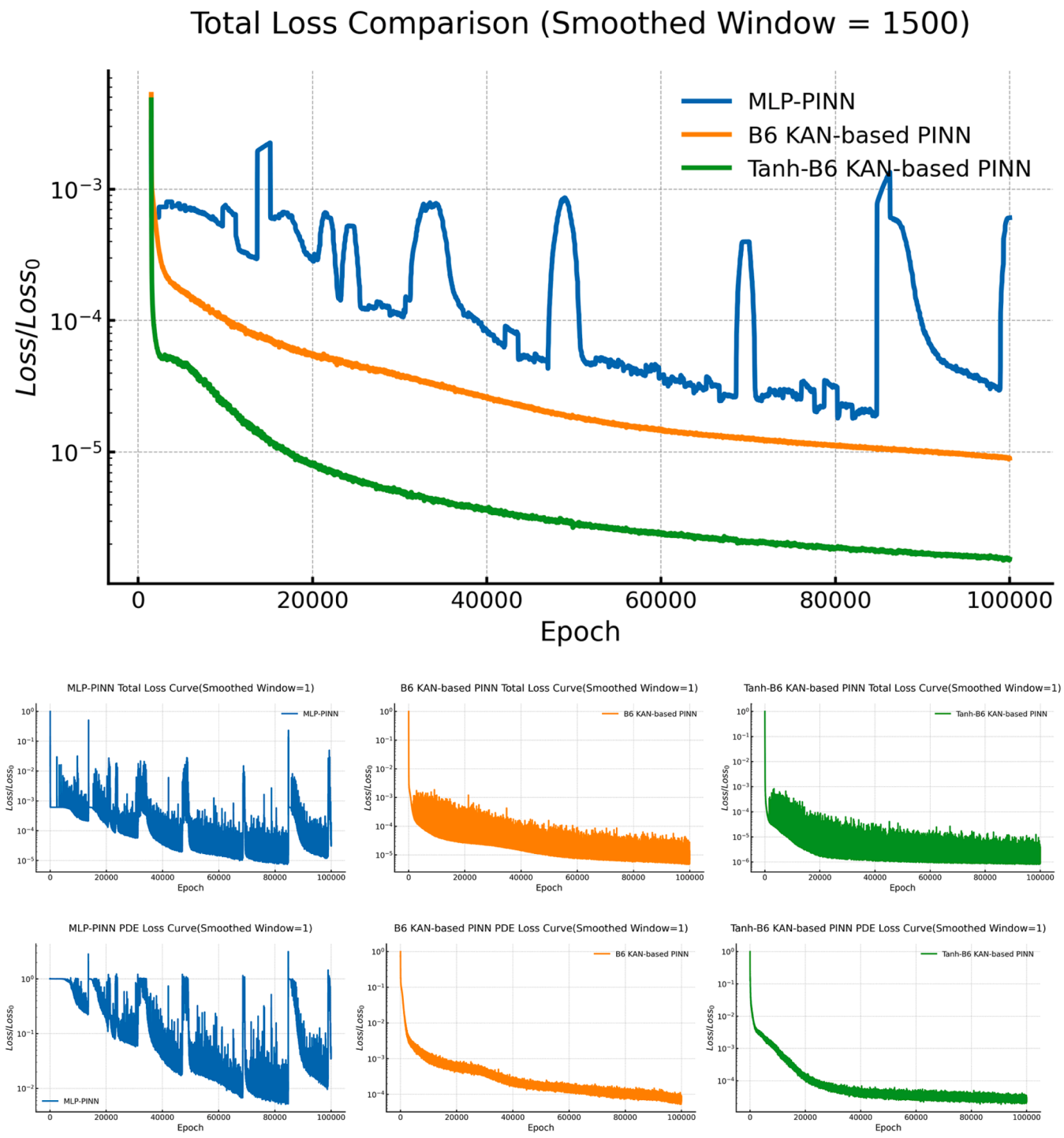


Fig. 21. PINNs loss curves for L-shaped thin plate.

edges under uniformly distributed load as an example, we employed Tanh-B6 KAN-based PINN neural network architectures with configurations of [2, 4, 1], [2, 5, 1], and [2, 6, 1] to investigate the impact of the number of neurons on convergence performance. As shown in Fig. 27(a), the convergence performances of the [2, 5, 1] and [2, 6, 1] architectures are similar, indicating that the use of 5 neurons is sufficient to meet computational requirements. Subsequently, the architectures [2, 5, 1], [2, 5, 5, 1], and [2, 5, 5, 5, 1] were used to investigate the impact of the number of hidden layers on the convergence performance. As shown in Fig. 27(b), the [2, 5, 5, 1] architecture exhibits the best convergence performance.

As shown in the experimental workflow, compared to the traditional MLP-PINN, the Tanh-B6 KAN-based PINN neural network architecture requires only a unit-level parameter adjustment each time, significantly simplifying the parameter optimization process. Additionally, varying the number of neurons and hidden layers has minimal impact on the convergence of the Tanh-B6 KAN-based PINN neural network.

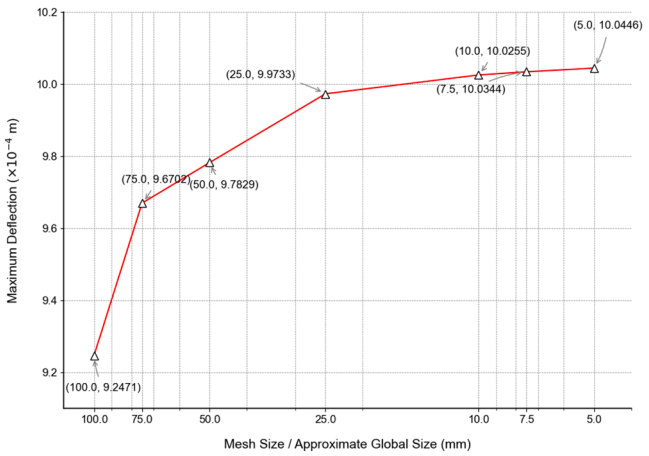


Fig. 22. Abaqus mesh convergence analysis.

6. Conclusions

In this paper, we proposed a Tanh-B6 KAN-based PINN framework to solve fourth-order Kirchhoff thin plate bending problems. By integrating Tanh activation functions with sixth-order B-splines, the method addresses computational efficiency and gradient instability issues encountered in traditional approaches. The main findings are summarized as follows:

(1) Superior Accuracy and Convergence: The proposed method reduces the relative L_2 error and Mean Absolute Error (MAE) by 1 to 3 orders of magnitude compared to MLP-PINN. It also achieves a lower loss floor and faster convergence than the standard B6 KAN.

Table 8
Deflection of L-shaped thin plate: PINNs solutions vs. Abaqus solutions ($\times 10^{-4}m$).

Methods of Solution	Max Deflection	Absolute Error of PINNs Solutions
Abaqus finite element	10.0446	/
MLP-PINN	7.2611	2.7835
B6 KAN-based PINN	8.4724	1.5722
Tanh-B6 KAN-based PINN	10.0079	0.0367

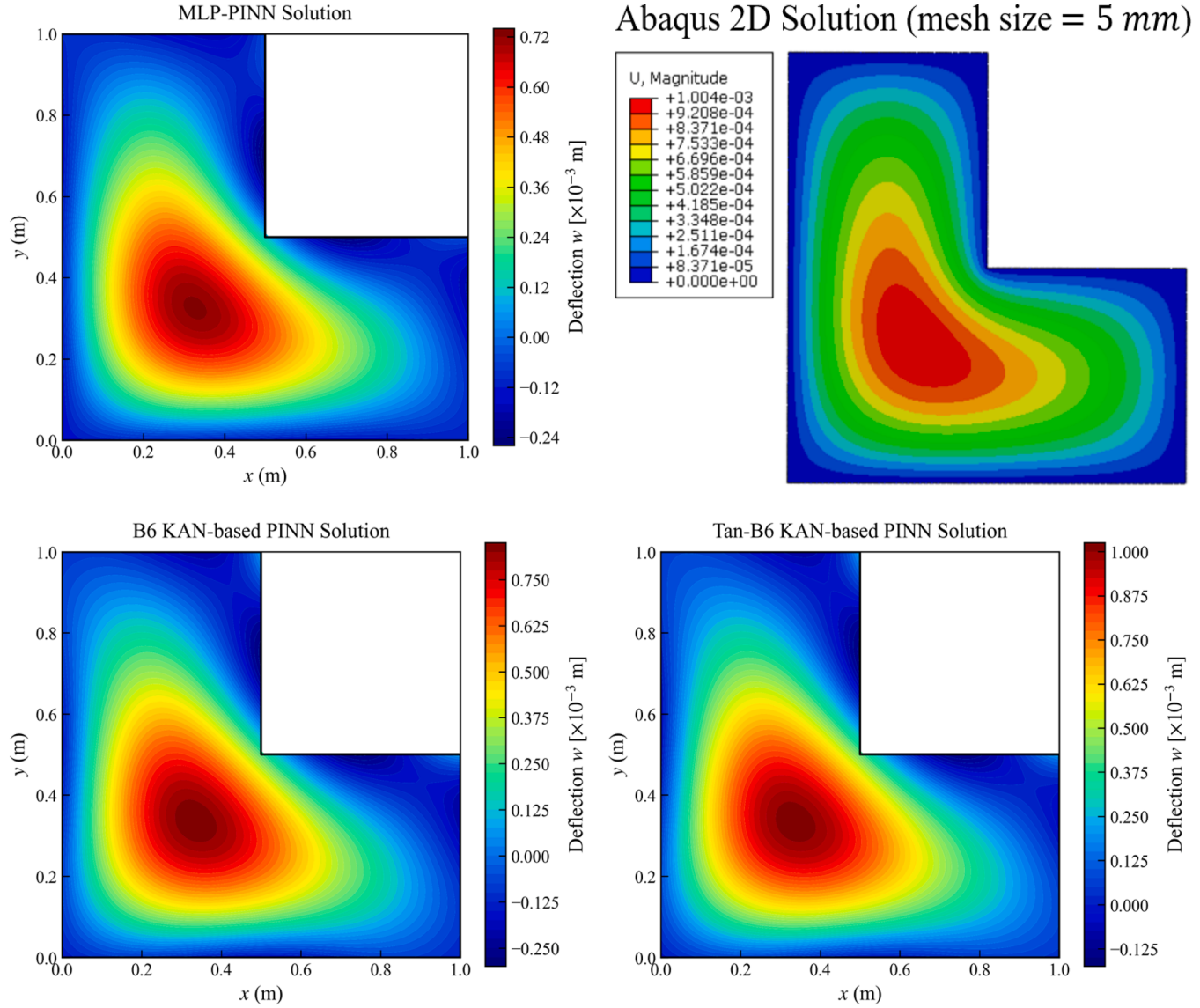


Fig. 23. 2D solutions for L-shaped thin plate.

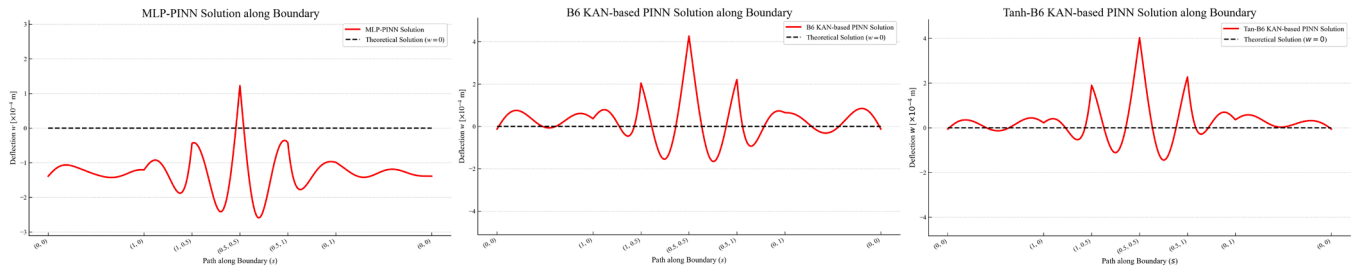


Fig. 24. Solutions along L-shaped boundary.

Table 9
Error statistics at the boundary of L-shaped thin plate.

Error ($\times 10^{-5}m$)	MLP-PINN	B6 KAN-based PINN	Tanh-B6 KAN-based PINN
Mean Absolute Error	12.9729	6.7171	5.3116
Root Mean Squared Error	13.6876	9.3607	8.2402
Max Absolute Error	25.9049	42.6309	40.2936

Table 10
PINNs solutions for inner corner point and nearby points in L-shaped thin plate ($\times 10^{-5}m$).

Points	MLP-PINN Solution	B6 KAN-based PINN Solution	Tanh-B6 KAN-based PINN Solution
(0.5,0.5)	12.2562	42.6307	40.2938
(0.5,0.51)	9.1590	39.2549	37.0676
(0.51,0.5)	9.2641	39.3353	37.1774

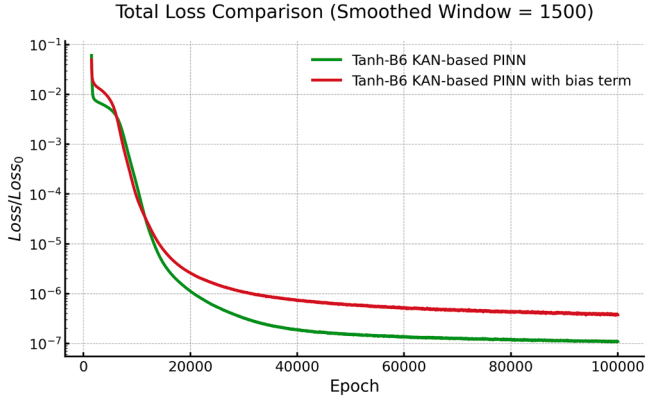


Fig. 25. Comparison of total loss curves with and without bias term.

(2) Parameter Efficiency: In contrast to deep MLP architectures, the Tanh-B6 KAN consistently yields high-precision results using a compact [2, 5, 5, 1] architecture. This reduces the number of trainable parameters by 2 to 3 orders of magnitude, significantly lowering computational costs.
(3) Robustness and Stability: The explicit node bias term proves unnecessary for this architecture. The method exhibits strong robustness, with convergence performance remaining stable despite variations in network depth or width.
(4) Mitigation of Spectral Bias: The C^4 continuity of the sixth-order B-splines ensures accurate calculation of high-order derivatives, effectively overcoming spectral bias and vanishing gradient issues inherent in high-order PDE problems.
(5) Applicability and Potential: The framework can be extended to other high-order partial differential equations, such as the Cahn-Hilliard equation in materials science, the biharmonic equation in fluid dynamics, and the Euler-Bernoulli beam problem.

Table 11
Error Statistics with and without bias term.

Error	Tanh-B6 KAN-based PINN	Tanh-B6 KAN-based PINN (with bias term)
Relative L_2 Norm Error	0.0121%	0.0924 %
Mean Absolute Error ($\times 10^{-5}m$)	0.1593	1.9415
Max Absolute Error ($\times 10^{-5}m$)	1.7124	4.3061

Table 12
Error Statistics at the Boundary with and without bias term.

Error ($\times 10^{-5}m$)	Tanh-B6 KAN-based PINN	Tanh-B6 KAN-based PINN (with bias term)
Mean Absolute Error	0.4360	1.9046
Root Mean Squared Error	0.5986	2.0990
Max Absolute Error	1.7044	4.2970

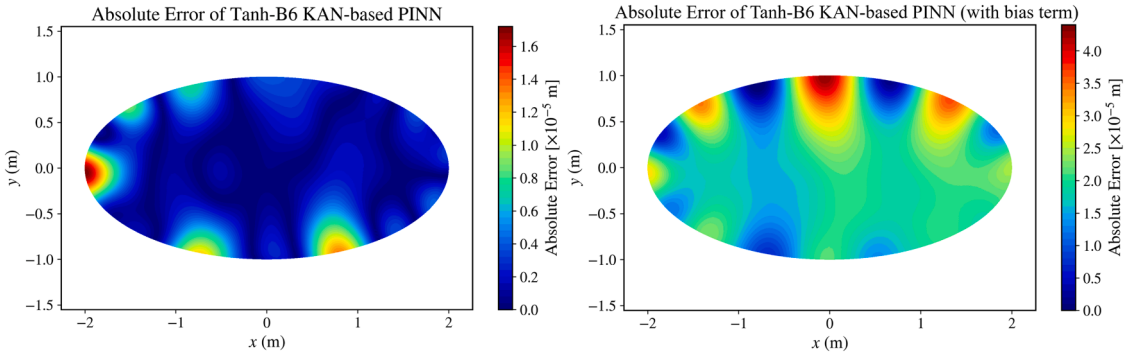
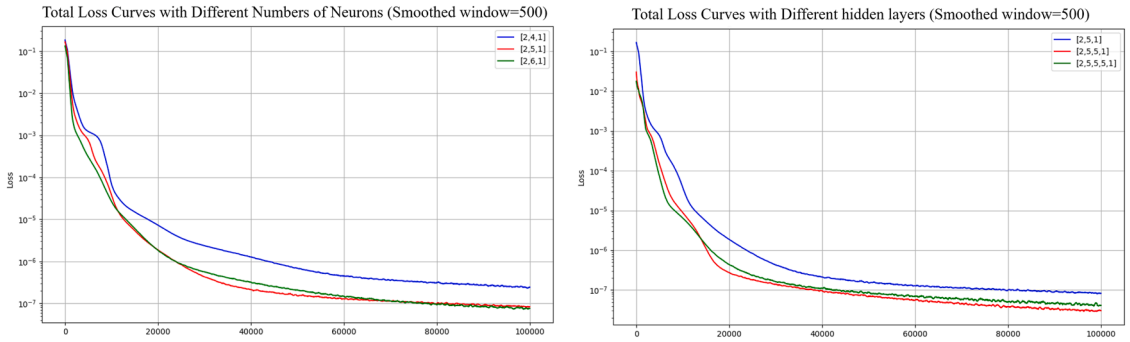


Fig. 26. Comparison of absolute errors with and without bias term.



(a) Different number of neurons. (b) Different number of hidden layers.

Fig. 27. Total loss curves of Tanh-B6 KAN-based PINN under different parameters.

Furthermore, due to its parameter efficiency, it holds potential value for solving inverse problems utilizing sparse data.

(6) Limitations and Future Work: A current limitation is that the method encounters challenges with local singularities in complex geometries, such as the re-entrant corners in L-shaped plates. Future work will address this issue by incorporating adaptive grid refinement or hybrid architectures.

CRediT authorship contribution statement

Shuo Liu: Writing – original draft, Visualization, Validation, Methodology, Conceptualization. **Yonghong Cao:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] Samaniego E, Anitescu C, Goswami S, Nguyen-Thanh VM, Guo H, Hamdia K, Zhuang X, Rabczuk T. An energy approach to the solution of partial differential equations in computational mechanics via machine learning: concepts, implementation and applications. Comput Methods Appl Mech Eng 2020;362:112790. https://doi.org/10.1016/j.cma.2019.112790.
[2] Zienkiewicz OC, Taylor RL. The finite element method set. Elsevier; 2005.
[3] Hughes TJ. The finite element method: linear static and dynamic finite element analysis. Courier Corporation; 2003.
[4] Bathe KJ. Finite element procedures. Klaus-Jurgen Bathe; 2006.
[5] Reddy JN. An introduction to the finite element method. New York 1993;27(14).
[6] Zhang X, Chen Z, Liu Y. The material point method: a continuum-based particle method for extreme loading cases. Academic Press; 2016.
[7] Liu GR. Meshfree methods: moving beyond the finite element method. CRC press; 2009.
[8] Rabczuk T, Belytschko T. Cracking particles: a simplified meshfree method for arbitrary evolving cracks. Int J Numer Methods Eng 2004;61(13):2316–43. https://doi.org/10.1002/nme.1151.
[9] Rabczuk T, Belytschko T. A three-dimensional large deformation meshfree method for arbitrary evolving cracks. Comput Methods Appl Mech Eng 2007;196(29–30):2777–99. https://doi.org/10.1016/j.cma.2006.06.020.
[10] Rabczuk T, Song JH, Zhuang X, Anitescu C. Extended finite element and meshfree methods. Academic Press; 2019.
[11] Nguyen VP, Rabczuk T, Bordas S, Duflot M. Meshless methods: a review and computer implementation aspects. Math Comput Simul 2008;79(3):763–813. https://doi.org/10.1016/j.matcom.2008.01.003.
[12] LeVeque RJ. Finite difference methods for ordinary and partial differential equations: steady-state and time-dependent problems. Soc Ind Appl Math 2007.
[13] Eymard R, Gallouët T. Herbin R. Finite volume methods. Handb Numer Anal 2000;7:713–1018. https://doi.org/10.1016/S1570-8659(00)07005-8.
[14] Brebbia CA, Telles JC, Wrobel LC. Boundary element techniques: theory and applications in engineering. Springer Science & Business Media; 2012.
[15] Chen S, Zhong X, Li Q, Wei X. Static and free vibration analyses of plates using a fully edge-based smoothed three-node Mindlin Plate element (FES-MIN3). Eng Anal Bound Elem 2025;179:106399. https://doi.org/10.1016/j.enganabound.2025.106399.
[16] Xia J, Chen S, Zhu P. Upper bound limit analysis of thin plates with a generalized conforming element formulated by the quadrilateral area coordinate method. Arch Appl Mech 2025;95(2):65. https://doi.org/10.1007/s00419-025-02771-1.
[17] Chen S, Dong H, Wei X, Liu F. A cell-based smoothed radial point interpolation method applied to lower bound limit analysis of thin plates. Eng Anal Boundary Elem 2025;172:106123. https://doi.org/10.1016/j.enganabound.2025.106123.
[18] Chen S, Xu M, Zhu X. A cell-based smoothed radial point interpolation method applied to kinematic limit analysis of thin plates. Eng Anal Bound Elem 2022;143:710–8. https://doi.org/10.1016/j.enganabound.2022.07.021.
[19] Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J Comput Phys 2019;378:686–707. https://doi.org/10.1016/j.jcp.2018.10.045.
[20] Sirignano J, Spiliopoulos K. DGM: a deep learning algorithm for solving partial differential equations. J Comput Phys 2018;375:1339–64. https://doi.org/10.1016/j.jcp.2018.08.029.
[21] Yu B. The deep Ritz method: a deep learning-based numerical algorithm for solving variational problems. Commun Math Stat 2018;6(1):1–2. https://doi.org/10.1007/s40304-018-0127-z.
[22] Wang Y, Sun J, Li W, Lu Z, Liu Y. CENN: conservative energy method based on neural networks with subdomains for solving variational problems involving heterogeneous and complex geometries. Comput Methods Appl Mech Eng 2022;400:115491. https://doi.org/10.1016/j.cma.2022.115491.
[23] Li W, Bazant MZ, Zhu J. A physics-guided neural network framework for elastic plates: comparison of governing equations-based and energy-based approaches. Comput Methods Appl Mech Eng 2021;383:113933. https://doi.org/10.1016/j.cma.2021.113933.
[24] Zhuang X, Guo H, Alajlan N, Zhu H, Rabczuk T. Deep autoencoder based energy method for the bending, vibration, and buckling analysis of Kirchhoff plates with transfer learning. Eur J Mech A Solids 2021;87:104225. https://doi.org/10.1016/j.euromechsol.2021.104225.
[25] Bastek JH, Kochmann DM. Physics-informed neural networks for shell structures. Eur J Mech A Solids 2023;97:104849. https://doi.org/10.1016/j.euromechsol.2022.104849.
[26] Wang W, Thai HT. A physics-informed neural network framework for laminated composite plates under bending. Thin-Walled Struct 2025;210:113014. https://doi.org/10.1016/j.tws.2025.113014.
[27] Wang L, Zhang L, He G. Chien-physics-informed neural networks for solving singularly perturbed boundary-layer problems. Appl Math Mech 2024;45(9):1467–80. https://doi.org/10.1007/s10483-024-3149-8.
[28] Peng LX, Sun JK, Tao YP, Huang ZM. Bending analysis of thin plates with variable stiffness resting on elastic foundation via a two-network strategy physics-informed neural network method. Structures 2024;68:107051. https://doi.org/10.1016/j.istruc.2024.107051. Elsevier.
[29] Zhang L, He G. Multi-scale-matching neural networks for thin plate bending problem. Theor Appl Mech Lett 2024;14(1):100494. https://doi.org/10.1016/j.taml.2024.100494.
[30] Kafkas P, Giannakopoulos G, Rekatsinas C. Solving linear elasticity problems using physics-informed neural networks. In: Proceedings of the 13th Hellenic Conference on Artificial Intelligence; 2024. p. 1–8. https://doi.org/10.1145/3688671.3688746.
[31] Liu Z., Wang Y., Vaidya S., Ruehle F., Halverson J., Soljačić M., Hou T.Y., Tegmark M. Kan: Kolmogorov-Arnold networks. arXiv preprint arXiv:2404.19756. 2024. https://doi.org/10.48550/arXiv.2404.19756.
[32] SS S., AR K., KP A. Chebyshev polynomial-based kolmogorov-arnold networks: an efficient architecture for nonlinear function approximation. arXiv preprint arXiv:2405.07200. 2024. https://doi.org/10.48550/arXiv.2405.07200.
[33] Shukla K, Toscano JD, Wang Z, Zou Z, Karniadakis GE. A comprehensive and FAIR comparison between MLP and KAN representations for differential equations and

operator networks. Comput Methods Appl Mech Eng 2024;431:117290. https://doi.org/10.1016/j.cma.2024.117290.
[34] Li Z. Kolmogorov-arnold networks are radial basis function networks. arXiv preprint arXiv:2405.06721. 2024. https://doi.org/10.48550/arXiv.2405.06721.
[35] Bozorgasl Z., Chen H. Wav-kan: wavelet kolmogorov-arnold networks, 2024. arXiv preprint arXiv:2405.12832. https://doi.org/10.48550/arXiv.2405.12832.
[36] Guo C, Sun L, Li S, Yuan Z, Wang C. Physics-informed Kolmogorov–Arnold network with Chebyshev polynomials for fluid mechanics. Phys Fluids 2025;37(9). https://doi.org/10.1063/5.0284999.
[37] Wang Y, Sun J, Bai J, Anitescu C, Eshaghi MS, Zhuang X, Rabczuk T, Liu Y. Kolmogorov–Arnold-Informed neural network: a physics-informed deep learning framework for solving forward and inverse problems based on Kolmogorov–Arnold Networks. Comput Methods Appl Mech Eng 2025;433:117518. https://doi.org/10.1016/j.cma.2024.117518.
[38] Bishop CM, Nasrabadi NM. Pattern recognition and machine learning. New York: springer; 2006.
[39] Franklin J. The elements of statistical learning: data mining, inference and prediction. Math Intell 2005;27(2):83–5. https://doi.org/10.1007/BF02985802.
[40] De Boor C, De Boor C. A practical guide to splines. New York: springer; 1978.
[41] Rahaman N, Baratin A, Arpit D, Draxler F, Lin M, Hamprecht F, Bengio Y, Courville A. On the spectral bias of neural networks. In: International conference on machine learning. PMLR; 2019. p. 5301–10. https://proceedings.mlr.press/v97/rahaman19a.html.
[42] Ruder S. An overview of gradient descent optimization algorithms. arXiv preprint arXiv:1609.04747. 2016. https://doi.org/10.48550/arXiv.1609.04747.
[43] Wang Y., Han X., Chang C.Y., Zha D., Braga-Neto U., Hu X. Auto-PINN: understanding and optimizing physics-informed neural architecture. arXiv preprint arXiv:2205.13748. 2022. https://doi.org/10.48550/arXiv.2205.13748.
[44] Yazdani A, Lu L, Raissi M, Karniadakis GE. Systems biology informed deep learning for inferring parameters and hidden dynamics. PLoS Comput Biol 2020;16(11):e1007575. https://doi.org/10.1371/journal.pcbi.1007575.
[45] Wang S, Teng Y, Perdikaris P. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. SIAM J Sci Comput 2021;43(5):A3055–81. https://doi.org/10.1137/20M1318043.
[46] Bischof R, Kraus MA. Multi-objective loss balancing for physics-informed deep learning. Comput Methods Appl Mech Eng 2025;439:117914. https://doi.org/10.1016/j.cma.2025.117914.
[47] Krishnapriyan A, Gholami A, Zhe S, Kirby R, Mahoney MW. Characterizing possible failure modes in physics-informed neural networks. Adv Neural Inf Process Syst 2021;34:26548–60. https://doi.org/10.48550/arXiv.2109.01050.
[48] Evans LC. Partial differential equations. American mathematical society; 2022.